Summer of Science 2026

Quantum Computing

Aaditya Vichare (25B2444)

Mentor: Sara Ahire

May 2026



Abstract

This report documents the early stages of the Summer of Science 2026 project on Quantum Computing. The first chapter develops the mathematical and physical foundations required for the rest of the work and connects them to introductory Qiskit implementations. The emphasis there is on building a self-contained understanding of the language of quantum computation, including complex vector spaces, linear operators, quantum postulates, composite systems, and the first circuit constructions used in the project. The second chapter focuses on Deutsch's algorithm as the simplest non-trivial quantum query algorithm, highlighting the role of phase kickback, oracle construction, and interference in achieving a single-query distinction between constant and balanced Boolean functions. Throughout, the treatment is formal but intentionally beginner-friendly, so that both the physics and the programming framework can be understood together and used as a basis for more advanced algorithms in later chapters.

Contents

1 Foundations and Introductory Circuit Demos in Qiskit **5**

1.1 Purpose of this chapter . . . 5
1.2 Mathematical foundations . . . 5
1.2.1 Vectors, state notation, and vector spaces . . . 5
1.2.2 Inner products and orthogonality . . . 6
1.2.3 Outer products and completeness . . . 7
1.2.4 Linear operators and matrices . . . 7
1.2.5 Adjoints, Hermitian operators, and unitarity . . . 8
1.2.6 Eigenvalues and diagonalization . . . 9
1.2.7 Positive operators and spectral structure . . . 9
1.2.8 Polar and singular decompositions . . . 10
1.2.9 Tensor products and composite systems . . . 10
1.2.10 Mixed states and density matrices . . . 11
1.2.11 Single-qubit gates and the Bloch sphere . . . 12
1.2.12 Controlled gates and multi-qubit universality . . . 13
1.3 Qiskit as a programming language . . . 13
1.4 Bell state entanglement demo . . . 14
1.4.1 Purpose of the script . . . 14
1.4.2 Building the circuit . . . 14
1.4.3 Quantum meaning of the gates . . . 15
1.4.4 Functions and methods used . . . 15
1.4.5 Why this script matters . . . 16
1.5 Quantum coin flip . . . 16
1.5.1 Purpose of the script . . . 16
1.5.2 Building the circuit . . . 16
1.5.3 Quantum meaning . . . 17
1.5.4 Functions and methods used . . . 17
1.5.5 Why this script matters . . . 18
1.6 Quantum random number generator . . . 18
1.6.1 Purpose of the script . . . 18

1.6.2 Building the circuit . . . 18
1.6.3 Functions and methods used . . . 19
1.6.4 Why this script matters . . . 20
1.7 Overall significance of the three programs . . . 20
1.8 Conclusion . . . 20

2 Deutsch's Algorithm **21**
2.1 Motivation and problem statement . . . 21
2.2 Oracle model and initial state preparation . . . 21
2.3 Phase kickback and effective phase oracle . . . 22
2.4 Interference and decision rule . . . 23
2.5 Oracle construction in the Qiskit implementation . . . 23
2.6 Exact statevector analysis . . . 24
2.7 Shot-based simulation and comparison . . . 25
2.8 Interpretation and connection to later work . . . 26

3 Teleportation Tests and Early Grover Study **27**
3.1 Quantum teleportation implementation . . . 27
3.1.1 Circuit structure and Bloch-state preparation . . . 27
3.1.2 Fidelity-based verification across Bloch states . . . 28
3.1.3 Visualisation and protocol diagram . . . 29
3.1.4 Strengths and suggested improvements . . . 29
3.2 Early Grover study and implementation . . . 30
3.2.1 Oracle and diffuser construction . . . 30
3.2.2 Full Grover circuit and optimal iteration count . . . 31
3.2.3 Amplitude tracking and visualisation . . . 32
3.2.4 Strengths and suggested improvements . . . 33

4 Full Grover Experiments and Noise-aware Extensions **35**
4.1 Extended Grover implementation . . . 35
4.1.1 Oracle and diffuser recapitulation . . . 35
4.1.2 Full Grover circuit and configuration layer . . . 36
4.1.3 Statevector amplitude tracking and analytic check . . . 37
4.2 Noise-aware Grover: simple Aer noise model . . . 38
4.2.1 Noise model design . . . 38
4.2.2 Ideal vs noisy counts and iteration sweeps . . . 38
4.2.3 Noise-strength scan at optimal iteration . . . 40
4.3 Single and multi-target experiments . . . 40
4.3.1 Experiment configurations . . . 40
4.3.2 Observed behaviour . . . 41

4.4 Capstone shaping 41

5 QFT, Phase Estimation, and Fidelity Analysis 43
5.1 Quantum Fourier Transform implementation 43
5.1.1 Manual QFT construction 43
5.1.2 QFT validation via fidelity checks 44
5.2 Basic quantum phase estimation 45
5.2.1 QPE circuit structure 45
5.2.2 Phase estimation and error measurement 45
5.2.3 QPE statevector overlap 46
5.3 Noise model and error analysis 47
5.3.1 Depolarizing + readout noise for QPE 47
5.4 Grover refinement and capstone alignment 47
5.4.1 Grover iteration trade-off near the optimum 47
5.4.2 Role in the Noise-aware Grover capstone 48

6 Noise-aware Grover Capstone 49
6.1 Grover implementation recap 49
6.1.1 Oracle and diffuser for single and multiple targets 49
6.1.2 Full Grover circuit and theoretical baseline 50
6.2 Noise model design 51
6.2.1 Depolarizing and readout errors 51
6.2.2 Ideal vs noisy simulators 51
6.3 Iteration sweeps and degradation analysis 52
6.3.1 Iteration sweep protocol 52
6.3.2 Ideal vs noisy success curves 53
6.3.3 Degradation summary metrics 53
6.4 Noise-strength sweeps and depth analysis 54
6.4.1 Noise-strength sweeps 54
6.4.2 Depth and CX-count correlation 54
6.5 Polished demonstration and capstone outcome 55
6.5.1 Polished demo structure 55
6.5.2 Key capstone conclusions 55

7 Quantum Sensing Applications in Astronomy, Navigation, and Materials Science 57
7.1 Quantum Sensing: Physical Foundations 57
7.1.1 Fragility as a Sensing Resource 57
7.1.2 Interference, Superposition, and Entanglement 58
7.1.3 Noise, Decoherence, and Quantum Control 58

7.2 Astronomical Interferometry and Quantum Enhancement 59
7.2.1 Classical Astronomical Interferometry 59
7.2.2 Quantum-Assisted Optical Interferometry 59
7.2.3 Squeezed Light and Gravitational-Wave Astronomy 60
7.2.4 Atom Interferometry for Gravitational-Wave Detection 60
7.3 Matter-Wave Interferometry and Space-Science Sensing 61
7.3.1 Atom Interferometers as Inertial and Gravity Sensors 61
7.3.2 Space-Based Atom Interferometry 62
7.3.3 China's Space-Based Cold Atom Gyroscope 63
7.4 Quantum Magnetometry and Navigation 63
7.4.1 Diamond NV Magnetometers 63
7.4.2 Magnetic-Field-Based Navigation 64
7.5 Quantum Sensing in Materials Science 64
7.5.1 NV-Based Nanoscale Sensing of Condensed Matter 64
7.5.2 Other Sensing Platforms 65
7.6 Long-Term Exploratory Angle: Asteroid-Mining-Related Space Applications 66
7.6.1 Asteroid Resource Prospecting 66
7.6.2 Potential Quantum-Sensing Contributions 66
7.7 Consolidation and Comparative Insights 67

A Implementation Notes: Refactored Deutsch's Algorithm Code 69
A.1 Overview of the code structure 69
A.2 Oracle metadata and factory 69
A.3 Deutsch circuit construction 71
A.4 Exact analysis via Statevector 72
A.5 Shot-based simulation with AerSimulator 73
A.6 Unified analysis and plotting 74

B Grover Amplitudes and Code-Level Tracking 76

C QFT and QPE Analytic Notes 79
C.1 Explicit QFT matrices for small n 79
C.1.1 One-qubit case ($n = 1$) 79
C.1.2 Two-qubit case ($n = 2$) 79
C.1.3 Three-qubit case ($n = 3$) 80
C.2 Binary-to-phase mapping in QPE 80
C.2.1 Phase encoding in QPE 80
C.2.2 Binary fraction interpretation 81
C.2.3 Phase error metric 81

Chapter 1

Foundations and Introductory Circuit Demos in Qiskit

1.1 Purpose of this chapter

This chapter establishes the formal tools needed for the rest of the project. The first part develops the mathematical language used in quantum mechanics and quantum computation, while the second part explains the three introductory Qiskit programs that were used to gain practical familiarity with superposition, entanglement, measurement, and random generation. The goal is not to reproduce the full textbook, but to retain the essential definitions, formulas, and interpretation needed later in the project.

1.2 Mathematical foundations

1.2.1 Vectors, state notation, and vector spaces

Quantum states are represented as vectors in a complex vector space. In Dirac notation, a vector is written as a ket, for example $|\psi\rangle$. For a qubit, the most useful basis is the computational basis $\{|0\rangle, |1\rangle\}$. Any qubit state can be written as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{1.1}$$

where $\alpha, \beta \in \mathbb{C}$.

The state must be normalized:

$$\langle\psi|\psi\rangle = 1. \tag{1.2}$$

For a qubit, this means

$$|\alpha|^2 + |\beta|^2 = 1. \tag{1.3}$$

Example. If

$$|\psi\rangle = \frac{1}{\sqrt{5}}|0\rangle + \frac{2}{\sqrt{5}}|1\rangle, \tag{1.4}$$

then

$$\left|\frac{1}{\sqrt{5}}\right|^2 + \left|\frac{2}{\sqrt{5}}\right|^2 = \frac{1}{5} + \frac{4}{5} = 1, \tag{1.5}$$

so the state is valid.

A set of vectors $\{|v_i\rangle\}$ is linearly independent if

$$\sum_i c_i|v_i\rangle = 0 \quad \Rightarrow \quad c_i = 0 \text{ for all } i. \tag{1.6}$$

A basis is a linearly independent spanning set. Every vector in the space can be uniquely written in terms of basis vectors.

Example. The qubit basis vectors satisfy

$$\langle 0|0\rangle = 1, \qquad \langle 1|1\rangle = 1, \qquad \langle 0|1\rangle = 0, \tag{1.7}$$

so they form an orthonormal basis.

1.2.2 Inner products and orthogonality

The inner product of two vectors $|v\rangle$ and $|w\rangle$ is written as $\langle v|w\rangle$. It provides the notion of overlap between states. The norm of a vector is defined as

$$\| \, |v\rangle \, \| = \sqrt{\langle v|v\rangle}. \tag{1.8}$$

If

$$\langle v|w\rangle = 0, \tag{1.9}$$

then the vectors are orthogonal.

In quantum computation, inner products determine measurement probabilities. This is one of the main reasons linear algebra is the natural mathematical framework for the subject.

Example. For basis vectors,

$$\langle 0|0\rangle = 1, \qquad \langle 1|1\rangle = 1, \qquad \langle 0|1\rangle = 0. \tag{1.10}$$

Thus $|0\rangle$ and $|1\rangle$ are orthogonal and normalized.

If a vector is written in an orthonormal basis as

$$|w\rangle = \sum_i w_i |i\rangle, \qquad |v\rangle = \sum_i v_i |i\rangle, \tag{1.11}$$

then

$$\langle w|v\rangle = \sum_i w_i^* v_i. \tag{1.12}$$

This is the matrix form of the inner product.

1.2.3 Outer products and completeness

The outer product $|v\rangle\langle w|$ is an operator. For any orthonormal basis $\{|i\rangle\}$, the completeness relation is

$$\sum_i |i\rangle\langle i| = I. \tag{1.13}$$

This identity is used constantly in quantum mechanics because it lets us expand states and operators in a basis-independent way.

Example. For a qubit,

$$|0\rangle\langle 0| + |1\rangle\langle 1| = I. \tag{1.14}$$

Applying this to any state leaves the state unchanged:

$$(|0\rangle\langle 0| + |1\rangle\langle 1|)\,|\psi\rangle = |\psi\rangle. \tag{1.15}$$

The outer product also acts as a projector when the same vector appears on both sides:

$$P = |v\rangle\langle v|. \tag{1.16}$$

Projectors are central in measurement theory and reduced-state descriptions.

1.2.4 Linear operators and matrices

A linear operator A satisfies

$$A\left(\sum_i c_i |v_i\rangle\right) = \sum_i c_i A|v_i\rangle. \tag{1.17}$$

This is the abstract version of matrix multiplication. In quantum computing, every gate is represented by a linear operator acting on the state vector.

If A is represented in a basis, then its matrix elements are the coefficients appearing in the expansion of the transformed basis vectors. This is why circuit simulation can be

done with matrices and state vectors.

Example. If a gate sends basis states to

$$A|0\rangle = |1\rangle, \qquad A|1\rangle = |0\rangle, \tag{1.18}$$

then the matrix form of the gate is

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \tag{1.19}$$

which is exactly the Pauli-X gate.

If $A : V \to W$ and $\{|v_i\rangle\}$, $\{|w_j\rangle\}$ are ordered bases of V and W, then

$$A|v_i\rangle = \sum_j A_{ji}|w_j\rangle, \tag{1.20}$$

and the numbers A_{ji} are the entries of the matrix representation of A.

1.2.5 Adjoints, Hermitian operators, and unitarity

The adjoint $A^\dagger$ is defined by

$$\langle w|A|v\rangle = \langle A^\dagger w|v\rangle. \tag{1.21}$$

An operator is Hermitian if

$$A^\dagger = A. \tag{1.22}$$

Hermitian operators correspond to observables.

An operator is unitary if

$$U^\dagger U = I. \tag{1.23}$$

Unitary operators preserve inner products and normalization. This is why closed quantum evolution and quantum gates are modeled by unitary matrices.

Example. If U is unitary and $|\psi\rangle$ is normalized, then

$$\langle\psi|U^\dagger U|\psi\rangle = \langle\psi|\psi\rangle = 1, \tag{1.24}$$

so $U|\psi\rangle$ is also a valid quantum state.

The projectors onto a subspace W spanned by an orthonormal basis $\{|v_1\rangle, \ldots, |v_k\rangle\}$ are

$$P = \sum_{i=1}^{k} |v_i\rangle\langle v_i|. \tag{1.25}$$

Such a projector is Hermitian and satisfies $P^2 = P$.

1.2.6 Eigenvalues and diagonalization

A vector $|v\rangle \neq 0$ is an eigenvector of A if

$$A|v\rangle = \lambda|v\rangle, \tag{1.26}$$

where λ is the eigenvalue. If a basis of eigenvectors exists, the operator is diagonalizable.

Example. The Pauli-Z gate acts as

$$Z|0\rangle = |0\rangle, \qquad Z|1\rangle = -|1\rangle. \tag{1.27}$$

So $|0\rangle$ and $|1\rangle$ are eigenvectors with eigenvalues $+1$ and -1, respectively.

The diagonalizable form of an operator can be written as

$$A = \sum_i \lambda_i |i\rangle\langle i|. \tag{1.28}$$

This form is especially useful for defining functions of operators.

1.2.7 Positive operators and spectral structure

An operator A is positive if

$$\langle v|A|v\rangle \geq 0 \tag{1.29}$$

for every vector $|v\rangle$. Positive operators appear naturally in measurement theory and density matrices.

If

$$A = \sum_i \lambda_i |i\rangle\langle i|, \tag{1.30}$$

then a function of A is defined spectrally as

$$f(A) = \sum_i f(\lambda_i) |i\rangle\langle i|. \tag{1.31}$$

Example. If A has eigenvalues 2 and 5, then A^2 has eigenvalues 4 and 25, and e^A has eigenvalues e^2 and e^5.

A useful special case is when $A^2 = I$. Then

$$e^{i\theta A} = \cos\theta\, I + i \sin\theta\, A. \tag{1.32}$$

This follows because the eigenvalues of A are ± 1, so the exponential acts as $e^{i\theta}$ and $e^{-i\theta}$ on the two eigenspaces.

Example. If $A = Z$, then

$$e^{i\theta Z} = \begin{pmatrix} e^{i\theta} & 0 \\ 0 & e^{-i\theta} \end{pmatrix} = \cos\theta\, I + i \sin\theta\, Z. \tag{1.33}$$

The trace of an operator is basis-independent because $\mathrm{tr}(AB) = \mathrm{tr}(BA)$ for matrices.

1.2.8 Polar and singular decompositions

The polar decomposition writes an operator as a product of a unitary and a positive operator:

$$A = UJ = KU, \tag{1.34}$$

where

$$J = \sqrt{A^\dagger A}, \qquad K = \sqrt{AA^\dagger}. \tag{1.35}$$

This decomposition separates the directional and scaling components of an operator.

The singular value decomposition states that for any square matrix A,

$$A = MDN, \tag{1.36}$$

where M and N are unitary and D is diagonal with positive entries.

Example. If a matrix stretches one basis direction by factor 3 and another by factor 1, then the singular values are 3 and 1, and the unitary factors encode the rotations before and after the stretching.

1.2.9 Tensor products and composite systems

The state space of a composite system is the tensor product of the component spaces:

$$\mathcal{H}_{AB} = \mathcal{H}_A \otimes \mathcal{H}_B. \tag{1.37}$$

If $|\psi\rangle \in \mathcal{H}_A$ and $|\phi\rangle \in \mathcal{H}_B$, then the joint state is

$$|\psi\rangle \otimes |\phi\rangle. \tag{1.38}$$

Example. If the first qubit is in $|0\rangle$ and the second is in $|1\rangle$, then the joint state is

$$|0\rangle \otimes |1\rangle = |01\rangle. \tag{1.39}$$

For two qubits, the basis states are $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

If A and B are operators on different systems, the combined action is given by

$$A \otimes B. \tag{1.40}$$

Example. If H acts on the first qubit and I on the second, then

$$(H \otimes I)|00\rangle = \frac{|00\rangle + |10\rangle}{\sqrt{2}}. \tag{1.41}$$

The inner product on tensor-product spaces satisfies

$$(\, |v_1\rangle \otimes |w_1\rangle,\ |v_2\rangle \otimes |w_2\rangle\,) = \langle v_1|v_2\rangle\langle w_1|w_2\rangle. \tag{1.42}$$

The Kronecker-product representation of matrices is

$$A \otimes B = \begin{pmatrix} A_{11}B & A_{12}B & \cdots \\ A_{21}B & A_{22}B & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}. \tag{1.43}$$

1.2.10 Mixed states and density matrices

A mixed state is an ensemble of pure states $\{|\psi_i\rangle\}$ with probabilities p_i. Its density matrix is

$$\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|. \tag{1.44}$$

A pure state is the special case

$$\rho = |\psi\rangle\langle\psi|. \tag{1.45}$$

Under unitary evolution,

$$\rho' = U\rho U^\dagger. \tag{1.46}$$

For a measurement described by operators M_m, the probability of outcome m is

$$p(m) = \mathrm{tr}(M_m^\dagger M_m \rho), \tag{1.47}$$

and the updated state is

$$\rho' = \frac{M_m \rho M_m^\dagger}{\mathrm{tr}(M_m^\dagger M_m \rho)}. \tag{1.48}$$

Example. If

$$\rho = \frac{1}{2}|0\rangle\langle 0| + \frac{1}{2}|1\rangle\langle 1|, \tag{1.49}$$

then ρ describes a 50-50 classical mixture of $|0\rangle$ and $|1\rangle$, not a superposition.

The reduced density matrix of a subsystem is obtained by partial trace:

$$\rho_A = \mathrm{tr}_B(\rho_{AB}). \tag{1.50}$$

1.2.11 Single-qubit gates and the Bloch sphere

Any single-qubit gate is represented by a unitary operator. A qubit can be written in Bloch-sphere form as

$$|\psi\rangle = e^{ix}\left(\cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle\right), \tag{1.51}$$

where the global phase e^{ix} is physically irrelevant.

The Pauli matrices are

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{1.52}$$

The rotation gates are

$$R_x(\theta) = e^{-i\theta X/2}, \qquad R_y(\theta) = e^{-i\theta Y/2}, \qquad R_z(\theta) = e^{-i\theta Z/2}. \tag{1.53}$$

Example. Using $X^2 = Y^2 = Z^2 = I$, we obtain

$$e^{-i\theta X/2} = \cos\frac{\theta}{2}\, I - i\sin\frac{\theta}{2}\, X, \tag{1.54}$$

and similarly for Y and Z.

Any unitary single-qubit gate can be decomposed as

$$U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta). \tag{1.55}$$

This is the most useful structural fact for single-qubit gates in the project.

The Hadamard and T gates are

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}. \tag{1.56}$$

Example. The Hadamard gate sends the basis state $|0\rangle$ to an equal superposition:

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}. \tag{1.57}$$

1.2.12 Controlled gates and multi-qubit universality

The controlled-NOT gate takes a control qubit and a target qubit. It flips the target if and only if the control is $|1\rangle$. In basis form:

$$|00\rangle \mapsto |00\rangle, \qquad |01\rangle \mapsto |01\rangle, \qquad |10\rangle \mapsto |11\rangle, \qquad |11\rangle \mapsto |10\rangle. \tag{1.58}$$

The CNOT gate is fundamental because, together with single-qubit gates, it can construct any multi-qubit unitary operation to arbitrary accuracy. A controlled-U gate can be decomposed using single-qubit gates and CNOTs.

Example. If a qubit is in state $|1\rangle$ and acts as a control, and another qubit is in $|0\rangle$, then after CNOT the target becomes $|1\rangle$. Thus the gate behaves like a quantum version of a conditional statement.

For a controlled-U gate with multiple controls, the notation $C^n(U)$ is used. The broader universality statement is that hadamard, phase, CNOT, and $\pi/8$ gates can approximate any quantum computation to arbitrary precision.

1.3 Qiskit as a programming language

Qiskit is the software layer used to express quantum circuits in code. In the present project, it is useful to think of Qiskit as a small language with its own syntax and standard objects. The central idea is to build a circuit, apply gates, and then simulate or measure the resulting state. Documentation and learning material for Qiskit and IBM Quantum are available through the official online resources [2, 3].

The main objects used in the attached scripts are (the codes are available in the GitHub repository [4]) :

- `QuantumRegister`: stores quantum bits.
- `ClassicalRegister`: stores measured classical bits.
- `QuantumCircuit`: combines quantum and classical registers into a circuit.
- `AerSimulator`: runs the circuit on a simulator.
- `Statevector`: gives the exact state of a circuit before measurement.
- `plot_histogram`: visualizes measurement statistics.
- `qc.draw(output="text")`: prints the circuit in text form.

The scripts in this chapter are good beginner examples because they use this same pattern repeatedly: create registers, build a circuit, apply gates, measure, run the simulator, and analyze the output.

1.4 Bell state entanglement demo

1.4.1 Purpose of the script

This script constructs the four Bell states using Hadamard, CNOT, X, and Z gates, then analyzes the resulting statevector and measurement statistics. It is the clearest introduction to entanglement in the project because the final states cannot be factored into single-qubit states.

1.4.2 Building the circuit

The function

```
build_bell_state(state)
```
(1.59)

creates the circuit for one of the four Bell states. The line

```
qr = QuantumRegister(2, name="q")
```
(1.60)

creates a two-qubit quantum register. The line

```
cr = ClassicalRegister(2, name="c")
```
(1.61)

creates a two-bit classical register for measurements. The circuit object is created by

```
qc = QuantumCircuit(qr, cr).
```
(1.62)

The state-selection logic uses X and Z gates:

- `qc.x(qr[0])` prepares the Ψ-type Bell states by flipping the first qubit.
- `qc.h(qr[1])` creates superposition on the control qubit.
- `qc.cx(qr[1], qr[0])` entangles the two qubits.
- `qc.z(qr[1])` changes the relative phase to generate the minus-sign Bell states.

1.4.3 Quantum meaning of the gates

Starting from $|00\rangle$, applying H to one qubit gives

$$\frac{|00\rangle + |10\rangle}{\sqrt{2}}. \tag{1.63}$$

Applying CNOT then gives

$$\frac{|00\rangle + |11\rangle}{\sqrt{2}}, \tag{1.64}$$

which is the Bell state $|\Phi^+\rangle$.

Example. If the code applies X before H and CNOT, the state is shifted into the Ψ-family, giving superpositions such as

$$\frac{|01\rangle + |10\rangle}{\sqrt{2}}. \tag{1.65}$$

1.4.4 Functions and methods used

The function

```
build_bell_state_no_measure(state)
```
(1.66)

constructs the same Bell state circuit without measurement. This is important because measurement would destroy the exact amplitudes. The method

```
Statevector.from_instruction(qc)
```
(1.67)

returns the exact amplitudes of the circuit.

The function

```
analyse_statevector(state)
```
(1.68)

prints the amplitude and probability for each basis state. For a Bell state, only two amplitudes are non-zero, each with magnitude $1/\sqrt{2}$.

The function

```
run_bell_experiment(state, shots)
```
(1.69)

runs a shot-based simulation using

```
simulator = AerSimulator()
```
(1.70)

and

```
job = simulator.run(qc, shots=shots).
```
(1.71)

The method `result()` extracts the simulation result, and

```
get_counts(qc)
```
(1.72)

returns a dictionary of measurement outcomes.

The function

```
analyse_correlations(counts, state, shots)
```
(1.73)

computes how often the measured outcomes are correlated or anti-correlated. This is the part of the script that explicitly demonstrates entanglement.

The function

```
plot_all_bell_states(shots)
```
(1.74)

creates a 2×2 grid of histograms using Matplotlib. The dictionary

```
all_outcomes = {"00":0, "01":0, "10":0, "11":0}
```
(1.75)

is used to ensure all basis outcomes appear on the plot even if some have zero count.

1.4.5 Why this script matters

This script is the first full example of using Qiskit to connect theory and experiment. It introduces state preparation, exact statevector inspection, probabilistic measurement, histogram visualization, and the role of entanglement in multi-qubit systems.

1.5 Quantum coin flip

1.5.1 Purpose of the script

This program demonstrates the simplest possible quantum random process. A single qubit is placed into a superposition and then measured, producing a 50-50 distribution between the two classical outcomes.

1.5.2 Building the circuit

The function

```
build_coin_flip_circuit()
```
(1.76)

creates one quantum register and one classical register:

```
qr = QuantumRegister(1, name="coin")
```
(1.77)

and

```
cr = ClassicalRegister(1, name="result").
```
(1.78)

The circuit is created by

```
qc = QuantumCircuit(qr, cr).
```
(1.79)

The line

```
qc.h(qr[0])
```
(1.80)

creates the equal superposition

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}}. \tag{1.81}$$

The line

```
qc.measure(qr, cr)
```
(1.82)

measures the qubit and stores the result in the classical bit.

1.5.3 Quantum meaning

Before measurement, the qubit is in a coherent superposition. After measurement, it collapses to one of the basis states. Since the state after the Hadamard gate has equal amplitude on $|0\rangle$ and $|1\rangle$, the measurement probabilities are

$$P(0) = P(1) = \frac{1}{2}. \tag{1.83}$$

Example. If the circuit is run 1000 times, one typically expects roughly 500 zeros and 500 ones, though not exactly equal because of statistical fluctuations.

1.5.4 Functions and methods used

The function

```
run_coin_flip(num_trials)
```
(1.84)

calls the circuit builder, creates an `AerSimulator`, and runs the circuit for a specified number of shots. The statement

```
counts = result.get_counts(qc)
```
(1.85)

returns the measurement frequencies.

The function

```
display_results(counts, num_trials)
```
(1.86)

extracts the frequencies for outcomes 0 and 1, prints them, and draws a histogram using

```
plot_histogram(readable, title=...).
```
(1.87)

The call to

```
plt.tight_layout()
```
(1.88)

improves the plot spacing, and

```
plt.savefig("quantum_coin_flip_histogram.png", dpi=150)
```
(1.89)

stores the figure as an image file.

1.5.5 Why this script matters

This is the simplest possible demonstration of quantum randomness and measurement collapse. It also introduces a standard Qiskit workflow: create circuit, simulate, extract counts, and visualize.

1.6 Quantum random number generator

1.6.1 Purpose of the script

This program generalizes the coin flip to multiple qubits so that an n-bit random string is generated in a single circuit. The output can then be interpreted as a uniformly random integer.

1.6.2 Building the circuit

The function

```
build_qrng_circuit(num_bits)
```
(1.90)

creates a quantum register and classical register of size `num_bits`:

```
qr = QuantumRegister(num_bits, name="q")
```
(1.91)

and

```
cr = ClassicalRegister(num_bits, name="bits").
```
(1.92)

The main line is

```
qc.h(range(num_bits)),
```
(1.93)

which applies a Hadamard gate to every qubit. This creates the uniform superposition

$$H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle. \tag{1.94}$$

The measurement line

$$\texttt{qc.measure(qr, cr)} \tag{1.95}$$

collapses the state to one of the basis states.

Example. For $n = 3$, the state becomes

$$\frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle). \tag{1.96}$$

Each outcome therefore has probability 1/8.

1.6.3 Functions and methods used

The function

$$\texttt{generate_random_number(num_bits)} \tag{1.97}$$

simulates the QRNG once and converts the resulting bitstring to an integer using

$$\texttt{int(bitstring, 2)}. \tag{1.98}$$

Here the second argument 2 tells Python to interpret the string as binary.

The function

$$\texttt{generate_random_in_range(min_val, max_val)} \tag{1.99}$$

uses rejection sampling. First it chooses a number of bits large enough to cover the range. Then it repeatedly samples until the raw value is less than the span of the desired interval. This avoids modulo bias.

The function

$$\texttt{run_uniformity_test(num_bits, num_samples)} \tag{1.100}$$

runs the circuit many times, converts bitstrings into integers, fills in missing bins with zero counts, and plots the resulting histogram. The line

$$\texttt{ax.axhline(expected, ...)} \tag{1.101}$$

draws the expected average frequency per bin, which helps check whether the distribution

is close to uniform.

1.6.4 Why this script matters

This script is important because it shows how multiple qubits can be used to generate more than a single random bit. It also introduces careful implementation details such as rejection sampling, bin counting, and histogram comparison with the expected distribution.

1.7 Overall significance of the three programs

The three code examples together cover the core ideas needed at the beginning of the project:

- superposition, demonstrated by the coin flip and QRNG;
- entanglement, demonstrated by the Bell states;
- measurement, demonstrated in all three programs;
- Qiskit workflow, demonstrated through circuit construction, simulation, and plotting.

They are intentionally simple, but they establish the exact interface between theory and implementation that will be used in later chapters.

1.8 Conclusion

This chapter developed the mathematical and physical framework needed for the rest of the report. It covered the linear-algebraic structure of quantum states, the postulates of quantum mechanics, tensor-product composition, and the essential one- and two-qubit gates. It then explained three introductory Qiskit programs in enough detail that both the physics and the programming methods are visible. With this foundation in place, later chapters can move on to more advanced circuits and algorithms while reusing the same notation and coding patterns.

Chapter 2

Deutsch's Algorithm

2.1 Motivation and problem statement

Deutsch's algorithm is one of the earliest examples of quantum advantage in a query problem and provides a compact, interference-based demonstration of how quantum computation can reduce the number of oracle calls required to learn a global property of a function [5, 6, 8]. The problem considered is the following. Let

$$f : \{0, 1\} \to \{0, 1\} \tag{2.1}$$

be a Boolean function promised to be either *constant* or *balanced*. The goal is to determine which of these two cases holds. Classically, in the worst case one must evaluate $f(0)$ and $f(1)$, requiring two queries. Deutsch's quantum algorithm achieves this with a single query by making careful use of superposition, phase kickback, and interference [5, 6].

The four possible functions in this one-bit setting are

$$f(x) = 0, \qquad f(x) = 1, \qquad f(x) = x, \qquad f(x) = 1 - x, \tag{2.2}$$

where the first two are constant and the latter two are balanced.

2.2 Oracle model and initial state preparation

As in the IBM Quantum presentation of Deutsch and Deutsch–Jozsa algorithms, the oracle is encoded as a unitary operator acting on two qubits [5, 6]. The query register holds the input x and an ancilla register enables the function value to be encoded as a controlled bit flip:

$$U_f \left|x\right\rangle \left|y\right\rangle = \left|x\right\rangle \left|y \oplus f(x)\right\rangle , \tag{2.3}$$

where $\oplus$ denotes addition modulo 2. This form makes it explicit that U_f is a reversible extension of the classical function f.

The Deutsch circuit begins in the state

$$|\psi_0\rangle = |0\rangle\,|1\rangle\,, \tag{2.4}$$

where the first qubit is the query and the second is the ancilla. A Hadamard gate is then applied to both qubits. Using

$$H\,|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad H\,|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle\,, \tag{2.5}$$

the joint state after the Hadamard layer is

$$\begin{aligned} |\psi_1\rangle &= (H\,|0\rangle)\,(H\,|1\rangle) \\ &= \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)\left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \\ &= \frac{1}{2}(|0\rangle + |1\rangle)(|0\rangle - |1\rangle). \end{aligned} \tag{2.6}$$

In particular, the ancilla is now in the state $|-\rangle$, which is an eigenstate of the Pauli-X gate. This choice is what turns the oracle's controlled bit flip into a phase kickback effect on the query qubit [7, 8].

2.3 Phase kickback and effective phase oracle

The key mechanism exploited by Deutsch's algorithm is phase kickback: when a controlled operation acts on an eigenstate of its target, the eigenvalue can be "kicked back" as a phase on the control [7]. In the present case,

$$X\,|-\rangle = -\,|-\rangle\,, \tag{2.7}$$

so a conditional X on the ancilla produces phases conditioned on the query value.

Applying the oracle U_f to the state $|\psi_1\rangle$ yields

$$U_f\,|x\rangle\,|-\rangle = |x\rangle\,|f(x) \oplus -\rangle = (-1)^{f(x)}\,|x\rangle\,|-\rangle\,, \tag{2.8}$$

where the last equality follows from the fact that flipping $|-\rangle$ multiplies it by -1. Therefore, after the oracle call the joint state is

$$|\psi_2\rangle = \frac{1}{\sqrt{2}}\left((-1)^{f(0)}\,|0\rangle + (-1)^{f(1)}\,|1\rangle\right)|-\rangle\,. \tag{2.9}$$

At this point the ancilla has factorized and the function values $f(0)$ and $f(1)$ appear as phases on the query qubit. This is often summarized by saying that, with the ancilla in $|-\rangle$, the bit-flip oracle acts as an effective phase oracle

$$|x\rangle \mapsto (-1)^{f(x)} |x\rangle \tag{2.10}$$

on the query, which is also the intuition used in the Deutsch–Jozsa generalization [6, 9].

2.4 Interference and decision rule

To convert the relative phase information into a measurable bit, a final Hadamard gate is applied to the query qubit. Using

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \qquad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}, \tag{2.11}$$

we obtain

$$\begin{aligned} |\psi_3\rangle &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} H|0\rangle + (-1)^{f(1)} H|1\rangle \right) |-\rangle \\ &= \frac{1}{2} \Big[\left((-1)^{f(0)} + (-1)^{f(1)} \right) |0\rangle + \left((-1)^{f(0)} - (-1)^{f(1)} \right) |1\rangle \Big] |-\rangle . \end{aligned} \tag{2.12}$$

Two cases arise:

1. If f is constant, then $f(0) = f(1)$. In this case the coefficient of $|1\rangle$ vanishes and the query qubit is deterministically $|0\rangle$.
2. If f is balanced, then $f(0) \neq f(1)$. In this case the coefficient of $|0\rangle$ vanishes and the query qubit is deterministically $|1\rangle$.

The measurement rule is therefore

$$f \text{ constant} \implies \text{measure } 0, \qquad f \text{ balanced} \implies \text{measure } 1. \tag{2.13}$$

The answer is obtained through interference: the phases $(-1)^{f(0)}$ and $(-1)^{f(1)}$ cause constructive interference on one computational basis state and destructive interference on the other, depending on whether the function is constant or balanced [5, 6, 8].

2.5 Oracle construction in the Qiskit implementation

In the implementation, the four admissible functions are represented as separate oracle circuits acting on two qubits. The query qubit is the control and the ancilla is the

target. Writing the oracle as a stand-alone subcircuit U_f matches the formal definition in Eq. (2.3) and simplifies reasoning about the algorithm [5, 6]. The constructions are:

$$U_{f(x)=0} : I, \tag{2.14}$$

$$U_{f(x)=1} : X \text{ on ancilla}, \tag{2.15}$$

$$U_{f(x)=x} : \mathrm{CNOT}(\text{query} \to \text{ancilla}), \tag{2.16}$$

$$U_{f(x)=1-x} : X_{\text{query}}\, \mathrm{CNOT}(\text{query} \to \text{ancilla})\, X_{\text{query}}. \tag{2.17}$$

The last form implements a control on $|0\rangle$ by conjugating the control qubit with Pauli-X gates before and after the CNOT. This is a standard pattern for constructing "zero-controlled" operations from conventional CNOT gates.

Structurally, the implementation separates the code into four layers:

1. *Oracle generation*, producing a `QuantumCircuit` that encodes U_f.
2. *Deutsch circuit construction*, which prepares $|0\rangle\,|1\rangle$, applies the Hadamard layer, calls the oracle once, and applies the final Hadamard on the query qubit.
3. *Exact state analysis*, where the pre-measurement state is analyzed using Qiskit's `Statevector` tools.
4. *Shot-based simulation and visualization*, where the circuit is simulated on `AerSimulator` and the resulting measurement statistics are compared with the exact predictions.

This layered design makes the conceptual roles of each part of the code transparent and prepares the ground for later extensions to Deutsch–Jozsa and other query algorithms.

2.6 Exact statevector analysis

To verify the interference behaviour at the state level, the Deutsch circuit is first constructed without measurements and passed to the `Statevector` simulator. Qiskit's statevector functionality allows one to compute the exact amplitudes and derived probabilities before any measurement is performed [10, 12]. In particular, one can extract the marginal distribution on the query qubit by tracing out or summing over the ancilla degrees of freedom.

For each oracle, the implementation computes the exact probabilities

$$P(q_0 = 0), \qquad P(q_0 = 1), \tag{2.18}$$

from the final statevector. For the constant oracles $f(x) = 0$ and $f(x) = 1$, the calculation confirms that $P(q_0 = 0) = 1$ and $P(q_0 = 1) = 0$. For the balanced oracles $f(x) = x$

and $f(x) = 1 - x$, the calculation instead yields $P(q_0 = 0) = 0$ and $P(q_0 = 1) = 1$. This matches the analytic derivation above and shows explicitly that the algorithm is deterministic at the level of ideal state evolution.

To deepen the understanding of phase kickback, the code also prints statevectors at key intermediate stages:

- after initialization to $|0\rangle\,|1\rangle$,
- after the Hadamard layer, producing $|+\rangle\,|-\rangle$,
- after the oracle, where the function values appear as phases on the query basis states,
- after the final Hadamard on the query qubit, where the amplitudes collapse to either $|0\rangle$ or $|1\rangle$ depending on the oracle.

Printing the complex amplitudes at each stage makes the interference pattern explicit and demonstrates how the relative phases are introduced and then converted into measurement probabilities.

2.7 Shot-based simulation and comparison

After the exact statevector analysis, the same circuits are executed on `AerSimulator` with a finite number of shots. Aer is Qiskit's high-performance simulator for quantum circuits and supports both ideal and noisy simulation modes [11]. In this context, it is used in an ideal setting to produce counts dictionaries of the form

$$\{\text{"0"} : N_0,\ \text{"1"} : N_1\}, \tag{2.19}$$

where N_0 and N_1 are the observed frequencies of measuring 0 and 1 on the query qubit over a fixed number of repetitions.

The code compares the dominant measurement outcome with the expected classification (constant vs. balanced) for each oracle. In the absence of noise and with sufficiently many shots, the empirical distribution concentrates near the ideal probabilities predicted by the statevector analysis. This two-level comparison - using exact state evolution and shot-based sampling - underscores that the quantum advantage arises from the interference structure of the circuit and is not just a numerical artifact of simulation.

From a broader project perspective, this structure is useful because it mirrors the pattern that will appear in later chapters: derive the ideal behaviour analytically, confirm it using exact statevector tools, and then explore how finite-sampling and noise affect the results when using realistic backends.

2.8 Interpretation and connection to later work

Conceptually, the most important lesson from this chapter is that the "speedup" of Deutsch's algorithm does not come from evaluating $f(0)$ and $f(1)$ separately in a naive parallel way, but from encoding information about both values into the phase of a single superposition and then designing a measurement basis in which that phase interferes constructively or destructively [5–7]. The carefully chosen ancilla state $|-\rangle$ turns the standard oracle U_f into an effective phase oracle, and the final Hadamard gate on the query qubit maps relative phase differences into deterministic measurement outcomes.

This pattern - phase encoding followed by interference and measurement - reappears in more advanced algorithms such as Deutsch–Jozsa, Simon's algorithm, and variants of Grover's algorithm [6, 8]. The refactored implementation developed in this chapter is therefore a useful stepping stone: it clarifies the role of the oracle, isolates the phase kickback mechanism, and introduces a rigorous method for comparing exact and sample-based simulations that will be reused in later chapters.

Chapter 3

Teleportation Tests and Early Grover Study

3.1 Quantum teleportation implementation

3.1.1 Circuit structure and Bloch-state preparation

The teleportation script implements the standard three-qubit protocol: Alice holds the message qubit q_0 and one half of a Bell pair q_1, Bob holds the other half q_2, and an arbitrary single-qubit state is teleported from q_0 to q_2 using two classical bits and local corrections. The state preparation helper

$$\texttt{prepare_state(qc, qubit, theta, phi)} \tag{3.1}$$

applies an R_y rotation followed by an R_z rotation,

$$\texttt{qc.ry}(\theta, \texttt{qubit}); \quad \texttt{qc.rz}(\phi, \texttt{qubit}), \tag{3.2}$$

which prepares the general Bloch-sphere state

$$|\psi(\theta,\phi)\rangle = \cos\frac{\theta}{2}\,|0\rangle + e^{i\phi}\sin\frac{\theta}{2}\,|1\rangle\,. \tag{3.3}$$

This parameterization matches the standard Bloch-sphere form used in textbooks and the IBM Quantum course material [1]. It allows the same circuit to be tested on computational-basis states $|0\rangle$, $|1\rangle$, equatorial states $|\pm\rangle$, and more general superpositions such as $|+i\rangle$.

The function

$$\texttt{build_teleportation_circuit(theta, phi)} \tag{3.4}$$

constructs the full three-qubit teleportation circuit with three classical bits. The logical

stages are:

1. State preparation on q_0 via `prepare_state`, with a barrier labelled "State prepared".

2. Bell-pair creation between q_1 and q_2 by applying H on q_1 followed by CNOT($q_1 \rightarrow q_2$), preparing $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$.

3. Alice's Bell-state measurement on (q_0, q_1) using CNOT($q_0 \rightarrow q_1$) and H on q_0, followed by measurements of q_0 and q_1 into classical bits c_1 and c_0, respectively.

4. Bob's classically controlled corrections, where `qc.x(q2)` is applied if $c_0 = 1$ and `qc.z(q2)` is applied if $c_1 = 1$, implemented using Qiskit's `if_test` control-flow feature.

5. A final measurement of q_2 into c_2 for demonstration runs.

The use of barriers makes the logical stages of the protocol clear in the drawn circuit and corresponds well to the conceptual steps described in standard teleportation expositions [1].

3.1.2 Fidelity-based verification across Bloch states

Beyond the direct three-bit measurement circuit, the code includes a dedicated fidelity-verification routine

$$\texttt{verify_teleportation(theta, phi, shots)}. \tag{3.5}$$

This function reconstructs a teleportation circuit that uses a `ClassicalRegister` of size two for Alice's measurement bits and leaves Bob's qubit unmeasured. It then uses

$$\texttt{AerSimulator(method="statevector")} \tag{3.6}$$

and

$$\texttt{qc.save_density_matrix(qubits=[2])} \tag{3.7}$$

to obtain the reduced density matrix of Bob's qubit after the classically controlled corrections. This approach follows the pattern recommended in Qiskit documentation and examples for extracting state information from a subset of qubits.

The target state is computed analytically as

$$\texttt{target_statevector(theta, phi)} = \begin{pmatrix} \cos(\theta/2) \\ e^{i\phi}\sin(\theta/2) \end{pmatrix}, \tag{3.8}$$

and the corresponding pure-state density matrix is formed as $|\psi\rangle\langle\psi|$. The script then uses `state_fidelity` from `qiskit.quantum_info` to compute the fidelity between Bob's reduced density matrix and this target. This is exactly the type of fidelity-based check used for teleportation and state-transfer demos in many Qiskit and IBM Quantum learning materials.

The list `TEST_STATES` includes six named test states:

- $|0\rangle$, $|1\rangle$ (computational basis),
- $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ (Hadamard eigenstates),
- $|+i\rangle = (|0\rangle + i\,|1\rangle)/\sqrt{2}$ (equatorial phase state),
- an arbitrary state with $\theta = \pi/3$, $\phi = \pi/4$.

The helper

```
run_all_states(shots)
```
(3.9)

loops over these tests, prints the fidelity for each case, and marks a test as passing if the fidelity exceeds 0.99. In ideal, noiseless simulation, the fidelities are observed to be numerically very close to 1.0, in line with theoretical expectations for perfect teleportation [1].

3.1.3 Visualisation and protocol diagram

Two plotting functions are included:

- `plot_protocol_diagram()`, which uses Matplotlib primitives to draw a schematic timeline of the teleportation protocol, with qubit lines, entanglement arrows, classical bit arrows, and labelled gate blocks.
- `plot_fidelities(results)`, which produces a bar chart of fidelity versus test state, highlighting that all chosen Bloch states are teleported with essentially unit fidelity.

3.1.4 Strengths and suggested improvements

The teleportation code already demonstrates several good practices:

- explicit state preparation on the Bloch sphere rather than only basis states;
- separation between the demonstration circuit (with three classical bits) and the fidelity-analysis circuits;
- use of `AerSimulator` in statevector mode with density-matrix extraction and `state_fidelity` for rigorous verification;

- inclusion of multiple test states and clear pass/fail criteria.

For later chapters, the following improvements would further tidy and generalise the code:

1. **Remove duplication between `build_circuit_no_measure` and `verify_teleportation`.** Both functions effectively reimplement the same teleportation logic with slightly different register sizes. Refactoring the core teleportation steps into a single helper that takes flags for measurement and for saving the density matrix would simplify maintenance and ensure that any change to the protocol is applied consistently.

2. **Extend Bloch-state sampling beyond a fixed test list.** For a more systematic verification, one could loop over a grid of θ and ϕ values (for instance, a coarse lattice on the Bloch sphere) and plot the distribution of fidelities. This would make the statement "teleportation succeeds for arbitrary input states" more visibly quantitative.

3. **Annotate code with references.** Adding short comments citing the standard teleportation circuit from Nielsen & Chuang or IBM resources would help connect the implementation back to the theory sources used in Chapter 1 and Chapter 3 [1].

4. **Add noise experiments as a bridge to the capstone.** Although noise modelling is planned for later chapters in the context of Grover, a small extension here could run the same fidelity checks under a basic depolarizing or amplitude-damping noise model on the entangling gates. This would provide an early example of how teleportation fidelity degrades in non-ideal conditions.

These changes are not necessary for correctness, but they align with the PoA emphasis on code improvements and on preparing a midterm-style discussion of strengths and limitations of the implementations.

3.2 Early Grover study and implementation

3.2.1 Oracle and diffuser construction

The Grover script implements an n-qubit search oracle and diffuser from first principles, in line with standard presentations of Grover's algorithm [1, 5, 6]. The oracle builder

$$\texttt{build_oracle(n_qubits, targets)} \tag{3.10}$$

constructs a phase oracle U_f such that

$$U_f \left|x\right\rangle = \begin{cases} -\left|x\right\rangle, & x \in \texttt{targets}, \\ \left|x\right\rangle, & \text{otherwise}, \end{cases} \tag{3.11}$$

by looping over each integer target and:

1. converting it to an n-bit binary string,
2. applying X gates to qubits where the target bit is 0 (to map the marked basis state to $|1\cdots1\rangle$),
3. applying a multi-controlled Z using the pattern $H \cdot \mathrm{MCX} \cdot H$ on the last qubit,
4. uncomputing the initial X gates to restore the computational basis.

This is the standard technique for implementing a multi-controlled phase flip based on Qiskit's multi-controlled-X gate (`mcx`) and is consistent with Qiskit tutorial implementations of custom Grover oracles.

The diffusion operator is built by

$$\texttt{build_diffuser(n_qubits)}, \tag{3.12}$$

which implements the inversion-about-the-average operation

$$D = 2\,|s\rangle\,\langle s| - I, \qquad |s\rangle = H^{\otimes n}\,|0\rangle^{\otimes n}\,, \tag{3.13}$$

via the circuit

$$D = H^{\otimes n}\,X^{\otimes n}\,Z_0\,X^{\otimes n}\,H^{\otimes n}, \tag{3.14}$$

where Z_0 denotes a phase flip on the all-zero state. In the code this is implemented exactly as

```
qc.h(range(n_qubits))
qc.x(range(n_qubits))
qc.h(n_qubits - 1)
qc.mcx(list(range(n_qubits - 1)), n_qubits - 1)
qc.h(n_qubits - 1)
qc.x(range(n_qubits))
qc.h(range(n_qubits))
```

which directly mirrors the usual textbook circuit for the Grover diffuser [1].

3.2.2 Full Grover circuit and optimal iteration count

The function

$$\texttt{optimal_iterations(n_qubits, n_targets)} \tag{3.15}$$

computes the approximate optimal number of Grover iterations

$$k \approx \frac{\pi}{4}\sqrt{\frac{N}{M}}, \qquad N = 2^n,\ M = \texttt{n_targets}, \tag{3.16}$$

rounds to the nearest integer, and clips at a minimum of 1. This formula is the standard asymptotic result for Grover's algorithm and is used widely in both analytic and numerical studies [1].

The full circuit builder

$$\texttt{build_grover_circuit(n_qubits, targets, n_iterations)}, \tag{3.17}$$

sets up the uniform superposition

$$H^{\otimes n}\left|0\right\rangle^{\otimes n}, \tag{3.18}$$

applies the oracle and diffuser in sequence for the designated number of iterations, and finally measures all qubits. Barriers labelled by iteration number are added for readability in the text drawing of the circuit. The simulation helper

$$\texttt{run_grover(n_qubits, targets, n_iterations, shots)} \tag{3.19}$$

then decomposes the circuit twice using `qc.decompose().decompose()` and executes it on `AerSimulator`, returning the measurement counts. The double decomposition step ensures that nested subcircuits (representing the oracle and diffuser) are fully broken down into native gates that the simulator can handle directly, a point also highlighted in various Qiskit Aer tutorials [11].

A simple helper

$$\texttt{success_probability(counts, targets, n_qubits, shots)} \tag{3.20}$$

converts these counts into a success probability by summing frequencies of all bitstrings corresponding to target integers and dividing by the total number of shots.

3.2.3 Amplitude tracking and visualisation

To study amplitude amplification in more detail, the script provides a function

$$\texttt{track_amplitudes(n_qubits, target, max_iters)}, \tag{3.21}$$

which uses Qiskit's `Statevector` class to track the exact probability of the marked state as a function of iteration count:

1. It prepares the initial uniform superposition state using a small circuit with Hadamards

and converts it to a `Statevector`.

2. For each iteration, it records the target and non-target probabilities via `sv.probabilities_dict()`, then evolves the state by one Grover step implemented as an appended oracle-plus-diffuser circuit, decomposed to primitive gates.

The resulting probability lists are plotted by

$$\texttt{plot_amplitude_amplification}, \tag{3.22}$$

which highlights the optimal iteration number k and shows that the target probability rises from $1/N$ to near unity and then oscillates, consistent with the standard sinusoidal analysis of Grover's algorithm [1, 5, 6].

The script also includes:

- `plot_measurement_histogram`, which plots the full distribution of measurement outcomes and highlights the target states in a different colour.
- `classical_vs_grover` and `plot_speedup_comparison`, which generate data and plots comparing classical query complexity $N/2$ with Grover's $(\pi/4)\sqrt{N}$ as functions of n, both in linear and log scale.

These plots provide the first steps toward the "scalability analysis" and complexity comparison that will be central to the capstone project on Noise-aware Grover.

3.2.4 Strengths and suggested improvements

The Grover implementation already covers:

- a general n-qubit oracle that supports single and multiple marked elements;
- a correct diffuser implementation and control over the number of iterations;
- statevector-based amplitude tracking for conceptual understanding;
- measurement-based success probability and visualisation for different iteration counts;
- explicit comparison of classical versus quantum query complexity.

To better align with later chapter objectives the following refinements may be made:

1. **Provide a simple analytic reference curve for amplitude amplification.** Since the probability of the target after k iterations can be written explicitly in terms of $\sin^2\big((2k+1)\theta\big)$ with $\sin^2\theta = M/N$, plotting this theoretical curve alongside the tracked probabilities would give a direct numerical check of the standard Grover analysis.

2. **Cleanly separate "study" mode from "demo" mode.** The current `__main__` section performs several tasks (single-target run, multi-target run, iteration sweep, and three plots). Introducing command-line flags or a small configuration section that turns individual analyses on or off would make the script easier to reuse for later experiments, especially when noise models are added.

3. **Introduce a minimal noise model early.** The PoA emphasises a Noise-aware Grover capstone. As a bridge, the current script could be extended with an optional noise flag that replaces `AerSimulator()` with a noise-configured simulator or uses the Aer primitives described in the latest Qiskit Aer guides [11]. The success probability curves with and without noise would then provide an early preview of the final capstone behaviour.

4. **Document resource scaling.** The existing scalability analysis focuses on query complexity. For a more code-centric perspective, it would be useful to log or record the number of two-qubit gates and the depth of the decomposed oracle and diffuser as a function of n. This would provide a concrete measure of circuit complexity alongside the abstract query-cost plots.

Chapter 4

Full Grover Experiments and Noise-aware Extensions

4.1 Extended Grover implementation

4.1.1 Oracle and diffuser recapitulation

The oracle

$$\texttt{build_oracle(n_qubits, targets)} \tag{4.1}$$

constructs a phase-flip operator U_f such that

$$U_f \left|x\right\rangle = \begin{cases} -\left|x\right\rangle, & x \in \texttt{targets}, \\ \left|x\right\rangle, & \text{otherwise}, \end{cases} \tag{4.2}$$

by converting each integer target into an n-bit string, applying X gates to qubits corresponding to zero bits to map the marked state to $|1\cdots1\rangle$, applying a multi-controlled Z (realised as $H \cdot \mathrm{MCX} \cdot H$ on the last qubit), and uncomputing the initial X gates. For the single-qubit edge case $n = 1$, the oracle reduces to a single Z gate on q_0. This construction follows the standard pattern used in many Grover implementations and Qiskit tutorials.

The diffuser

$$\texttt{build_diffuser(n_qubits)} \tag{4.3}$$

implements inversion about the average,

$$D = 2\left|s\right\rangle\left\langle s\right| - I, \qquad \left|s\right\rangle = H^{\otimes n}\left|0\right\rangle^{\otimes n}, \tag{4.4}$$

via the well-known decomposition

$$D = H^{\otimes n} X^{\otimes n} Z_0 X^{\otimes n} H^{\otimes n}, \tag{4.5}$$

where Z_0 is a phase flip on the all-zero state. In the circuit, this is realised as:

1. H on all qubits followed by X on all qubits;
2. a multi-controlled Z on the all-one state (again via $H \cdot \mathrm{MCX} \cdot H$ on the last qubit, or a single Z when $n = 1$);
3. the inverse sequence of X and H gates.

This diffuser matches the textbook description of Grover's operator as a reflection about $|s\rangle$, and it is compatible with the analytic amplitude amplification picture discussed in the chapter 3 appendix.

4.1.2 Full Grover circuit and configuration layer

The full Grover circuit is constructed by

`build_grover_circuit(n_qubits, targets, n_iterations, measure, add_barriers)`. (4.6)

This helper wraps the oracle and diffuser into a configurable circuit:

- validation ensures that `targets` are non-empty, unique, and within the range $\{0, \ldots, 2^n - 1\}$;
- if `n_iterations` is `None`, the code uses

  $$k_{\text{opt}} \approx \frac{\pi}{4}\sqrt{\frac{N}{M}}, \tag{4.7}$$

  where $N = 2^n$ and $M =$ `len(targets)`, consistent with the analytic Grover analysis and with IBM's learning material on iteration choice;
- a `measure` flag toggles the inclusion of a classical register and final measurement, allowing the same builder to be used both for statevector-based analysis and for shot-based experiments;
- an `add_barriers` flag controls whether barriers are inserted between iterations to improve drawn-circuit readability.

This separation into core circuit builder plus configuration layer makes it easier to run a variety of experiments (single-target, multi-target, ideal, noisy) using the same underlying construction, which is important for the capstone where multiple parameter sweeps are envisaged.

4.1.3 Statevector amplitude tracking and analytic check

To connect with the analytic Grover amplitude analysis, chapter 4 introduces two helpers:

`grover_angle(n_qubits, n_targets)`, `theoretical_success_probability(n_qubits, n_target` (4.8)

The first computes the effective Grover angle

$$\theta = \arcsin \sqrt{\frac{M}{N}}, \tag{4.9}$$

while the second returns

$$P_{\text{marked}}(k) = \sin^2\big((2k+1)\theta\big), \tag{4.10}$$

the analytic marked-state probability after k iterations, as derived in the chapter 3 appendix.

The function

`track_amplitudes_single_target(n_qubits, target, max_iters)` (4.11)

uses Qiskit's `Statevector` class to compute the exact target probability after each iteration and compares it to the analytic curve:

1. It prepares the initial uniform superposition and converts it to a statevector.
2. For each k up to `max_iters`, it records the exact probability of the target bitstring from `sv.probabilities_dict()`, computes the theoretical value $\sin^2((2k+1)\theta)$, and stores both.
3. It then applies one Grover step (oracle plus diffuser) by evolving the statevector under a decomposed circuit.

This yields three sequences: empirical target probability, empirical non-target probability, and theoretical target probability. The plotting helper

`plot_amplitude_tracking(n_qubits, target, out_file)` (4.12)

then produces a figure where the numerical and analytic curves overlay closely, confirming that the implementation faithfully reproduces the textbook amplitude amplification behaviour under ideal conditions.

4.2 Noise-aware Grover: simple Aer noise model

4.2.1 Noise model design

To begin exploring the capstone's Noise-aware Grover direction, chapter 4 introduces a simple, tunable noise model implemented using Qiskit Aer's `noise` module. In particular, the script defines a small configuration structure

$$\texttt{NoiseConfig(p1, p2, p_meas, enabled)}, \tag{4.13}$$

where:

- `p1` is the single-qubit depolarizing probability,
- `p2` is the two-qubit depolarizing probability,
- `p_meas` is the classical readout error probability,
- `enabled` toggles noise on and off.

The function

$$\texttt{build_simple_noise_model(cfg)} \tag{4.14}$$

creates a `NoiseModel` instance and adds:

1. depolarizing noise with probability `p1` to a set of basic one-qubit gates (`"x"`, `"h"`, `"z"`, `"sx"`, `"rz"`);
2. depolarizing noise with probability `p2` to all `"cx"` gates;
3. a symmetric readout error described by a `ReadoutError` matrix with flip probability `p_meas`.

This pattern closely follows the Qiskit Aer documentation on building custom noise models using `depolarizing_error` and `ReadoutError` applied to gate and measurement operations.

The model is intentionally simple: it is not hardware-calibrated, but it is sufficient for demonstrating how Grover's success probability is reduced by generic gate and measurement errors.

4.2.2 Ideal vs noisy counts and iteration sweeps

The helper

$$\texttt{run_counts(n_qubits, targets, n_iterations, shots, noise_cfg)} \tag{4.15}$$

wraps circuit execution under both ideal and noisy conditions:

- it builds the appropriate Grover circuit with measurement,
- chooses either a noiseless `AerSimulator()` or a noisy `AerSimulator(noise_model=...)` depending on `noise_cfg`,
- transpiles the circuit for the chosen backend,
- runs it and returns the resulting counts.

Given counts and a list of targets, the success probability is obtained by summing all counts corresponding to target bitstrings and dividing by the total number of shots.

To study how success probability varies with the number of iterations, chapter 4 adds

$$\texttt{iteration_sweep(n_qubits, targets, max_iters, shots, noise_cfg)}, \tag{4.16}$$

which records for each iteration k:

- the theoretical marked-state probability $\sin^2((2k+1)\theta)$;
- the exact ideal probability from `exact_target_probability`, computed via a statevector;
- the sampled success probability from shot-based simulation, either ideal or noisy depending on `noise_cfg`.

Plotting functions then overlay these curves:

- analytic theory;
- ideal sampled probabilities (noiseless simulation);
- noisy sampled probabilities (with depolarizing and readout noise).

Empirically, the plots show that:

1. in the ideal case, the sampled success probabilities follow the analytic curve closely and peak near the expected optimal iteration count k_{opt};
2. in the noisy case, the peak success probability is noticeably lower and the optimal iteration count can shift slightly, reflecting the well-known fact that noise changes the effective rotation angle and reduces the maximum achievable amplitude.

This is a first concrete demonstration of Noise-aware Grover within the project.

4.2.3 Noise-strength scan at optimal iteration

To connect more directly to the capstone question "how does success probability degrade with noise strength?" the chapter 4 script also performs a noise-strength scan at fixed iteration count. For a chosen configuration (e.g. $n = 4$, single target, $k = k_{\text{opt}}$), it evaluates the success probability as a function of the two-qubit depolarizing parameter `p2` over a small grid. The single-qubit depolarizing probability `p1` and measurement error `p_meas` are tied to `p2` by simple proportionality to keep the search space small.

The resulting data are plotted by `plot_noise_strength_scan`, which shows how the success probability at nominally optimal iteration decreases as `p2` increases. Although the scan is limited to a small set of noise strengths, it already illustrates the qualitative behaviour reported in more detailed studies of Grover under depolarizing and related noise models: performance degrades significantly as the database size or noise level grows, and there exists a threshold beyond which the algorithm ceases to outperform classical search.

4.3 Single and multi-target experiments

4.3.1 Experiment configurations

Chapter 4 defines an explicit list of experiment configurations through a small `ExperimentConfig` dataclass:

- single-target search on $n = 4$ qubits (one marked basis state out of 16);
- multi-target search on $n = 4$ qubits (two marked states, e.g. $\{3, 11\}$);
- single-target search on $n = 5$ qubits (one marked state out of 32).

For each configuration, the script prints:

- the search-space size $N = 2^n$ and the list of targets (both as integers and bitstrings);
- the optimal iteration estimate k_{opt} and the classical average query count $\approx N/2$;
- the approximate Grover query scale $(\pi/4)\sqrt{N/M}$.

It then runs:

1. an ideal experiment (no noise) at k_{opt}, reporting success probability and the top few bitstrings in the measurement histogram;
2. a noisy experiment with a baseline noise configuration, again reporting success probability and the top bitstrings;

3. basic circuit metrics (depth, number of qubits, CX-count, and total one-qubit gate count) for the Grover circuit at k_{opt}.

These metrics will be useful later in the capstone discussion, where circuit depth and two-qubit gate counts are directly related to the amount of noise accumulated during the algorithm.

4.3.2 Observed behaviour

Under ideal simulation, the experimental results match expectations from Grover's theory:

- for single-target search, the marked basis state dominates the measurement histogram after k_{opt} iterations, with success probabilities close to unity for the small n used in chapter 4 experiments;
- for the multi-target case, success is defined as measuring any of the marked states, and the combined success probability peaks near the value predicted by the analytic amplitude amplification formula with $M = 2$.

When the simple depolarizing-plus-readout noise model is enabled, the marked states still appear more frequently than non-marked states, but:

- the peak success probability at k_{opt} is reduced, sometimes substantially;
- the measurement histogram shows additional weight on non-target states;
- deeper circuits (larger n or larger k) experience more severe degradation, reflecting the cumulative effect of gate noise and readout errors.

These trends mirror those reported in more detailed numerical and experimental evaluations of Grover on noisy simulators and real hardware. They also provide a concrete empirical foundation on which the Noise-aware Grover capstone will build.

4.4 Capstone shaping

Overall, chapter 4 fulfills its objectives in three ways. First, it delivers a full Grover implementation with a clean separation between oracle, diffuser, and configurable Grover circuit construction, suitable for both single- and multi-target problems. Second, it extends the implementation into a systematic experimental framework, including:

- statevector-based amplitude tracking and analytic checks,
- ideal single- and multi-target search at the predicted optimal iteration count,

- basic circuit metrics that relate gate counts and depth to potential noise accumulation.

Third, it introduces and exercises a simple, tunable noise model in Qiskit Aer, and uses it to:

- compare ideal and noisy success probabilities as a function of iteration count;
- quantify how success probability at the nominally optimal iteration degrades as the two-qubit depolarizing probability `p2` is increased.

These steps mark the transition from "Grover works in the ideal textbook setting" to "Grover behaves in a measurable way under specific noise assumptions," which is exactly the conceptual shift required for the Noise-aware Grover capstone. In chapters 5 and 6, this framework can be extended by:

- refining the noise model (e.g. including amplitude-damping or hardware-derived noise parameters);
- broadening the iteration and noise-strength sweeps;
- and formalising quantitative measures (e.g. fidelity or average success probability) to summarise how Grover's performance scales with both database size and noise strength.

This will culminate in a more complete answer to the capstone question: how does the success probability of Grover's search degrade with iteration count and noise, and where does the algorithm cease to provide a meaningful advantage over classical search in realistic noisy settings?

Chapter 5

QFT, Phase Estimation, and Fidelity Analysis

5.1 Quantum Fourier Transform implementation

5.1.1 Manual QFT construction

The quantum Fourier transform on n qubits is the unitary map

$$\mathrm{QFT}_n : |x\rangle \mapsto \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k/2^n} |k\rangle , \tag{5.1}$$

which can be decomposed into a sequence of Hadamard and controlled-phase gates followed by a bit-reversal (swap) layer [1]. In Qiskit, this decomposition is implemented directly by the function

```
build_manual_qft(n_qubits, do_swaps, inverse),
```
(5.2)

which uses two helper routines:

- `apply_qft_rotations`, a recursive function that performs:
  1. a Hadamard on the current target qubit;
  2. a ladder of controlled-phase gates $\mathrm{CP}(\pi/2^k)$ from earlier qubits onto that target;
  3. a recursive call on the remaining qubits.
- `swap_register`, which applies the final bit-reversal by swapping qubits 0 and $n-1$, 1 and $n-2$, and so on.

When `inverse=False`, the function builds the forward QFT. When `inverse=True`, it composes the inverse of the forward circuit (optionally including swaps) and renames

it as QFT†. This implementation follows the standard gate-level structure described in Qiskit's QFT documentation and in standard textbooks.

5.1.2 QFT validation via fidelity checks

To ensure correctness, chapter 5 includes several fidelity-based validation steps, using Qiskit's `Statevector` class and `state_fidelity` function:

1. **Identity check** $QFT^\dagger QFT \approx I$: the function

   `validate_qft_identity(n_qubits)` (5.3)

   constructs a circuit that applies the manual QFT followed by its inverse and compares the resulting state with the initial $|0\rangle^{\otimes n}$ via state fidelity. For small n, the observed fidelity is numerically very close to 1, confirming that $QFT^\dagger QFT$ acts as the identity in the noiseless simulator.

2. **Manual vs library QFT overlap**: the function

   `compare_manual_vs_library_qft(n_qubits)` (5.4)

   builds two circuits, one applying the manual QFT and one applying Qiskit's built-in `QFT` circuit with the same `do_swaps` setting, and computes the fidelity between their output statevectors. This tests that the hand-built decomposition matches the semantics of the library implementation documented in IBM's QFT API.

3. **Reconstruction fidelity on basis states**: for a chosen basis index x, the function

   `qft_reconstruction_fidelity(n_qubits, basis_index)` (5.5)

   prepares $|x\rangle$, applies the manual QFT followed by its inverse, and compares the resulting state to the original. The fidelity is again observed to be extremely close to 1 for the tested sizes, indicating that the forward and inverse transforms cancel as expected.

Together, these checks provide a compact yet robust validation of the manual QFT implementation. They leverage standard statevector and fidelity tools in Qiskit, in line with recommended practice for verifying custom circuits.

5.2 Basic quantum phase estimation

5.2.1 QPE circuit structure

Quantum phase estimation (QPE) is used to estimate eigenphases ϕ of a unitary U such that

$$U|u\rangle = e^{2\pi i\phi}|u\rangle, \tag{5.6}$$

by writing an approximation to ϕ onto a register of "counting" qubits via controlled powers of U and an inverse QFT [1]. In chapter 5, a simple QPE instance is implemented for a unitary that acts as a phase gate on an eigenstate $|1\rangle$, using the function

```
build_qpe_circuit(n_count, phase, eigenstate, measure, use_manual_iqft).
```
(5.7)

The circuit structure is:

1. Allocate n_{count} counting qubits and one target qubit.
2. Prepare the eigenstate: for $|1\rangle$, apply an X gate to the target; for $|0\rangle$, leave it unchanged.
3. Apply a Hadamard to each counting qubit to create a uniform superposition.
4. For each counting qubit j, apply a controlled phase gate with total phase $2\pi\phi 2^j$ onto the target, corresponding to controlled powers of U.
5. Apply an inverse QFT on the counting register, either via the manual implementation or via Qiskit's library QFT with `inverse=True`, as selected by `use_manual_iqft`.
6. If `measure=True`, measure the counting qubits to obtain a bitstring estimate of ϕ.

This is precisely the canonical QPE structure described in textbook treatments and in Qiskit's QPE tutorials, where controlled U^{2^j} gates and inverse QFT form the core of the algorithm.

5.2.2 Phase estimation and error measurement

To test QPE, the script uses

```
run_qpe(n_count, phase, shots, noise_cfg, label),
```
(5.8)

which:

- builds the QPE circuit for a given phase ϕ and counting-qubit count n_{count};

- runs it on either an ideal `AerSimulator` or a noisy simulator (see Section 5.3) depending on `noise_cfg`;
- collects counts and identifies the dominant outcome bitstring;
- converts that bitstring to an estimated phase $\hat{\phi} \in [0, 1)$ by interpreting it as a binary fraction;
- computes the absolute phase error $|\hat{\phi} - \phi|$, folded modulo 1 to handle wrap-around.

Results are collected for a small set of test phases (e.g. $\phi = 1/8, 1/4, 3/8, 5/8$), using three counting qubits so that phases of the form $k/8$ are exactly representable. For each phase, the script prints:

- the true phase ϕ,
- the ideal estimated phase $\hat{\phi}_{\text{ideal}}$ and its absolute error,
- the noisy estimated phase $\hat{\phi}_{\text{noisy}}$ and its absolute error.

In the ideal case, the estimated phases match the true phases to within numerical precision, as expected from the theory of QPE for exactly representable phases [1]. In the noisy case, the error increases and the dominant-outcome probability decreases, providing a quantitative measure of how noise affects phase estimation.

5.2.3 QPE statevector overlap

To further validate the QPE implementation, the function

```
qpe_statevector_overlap(n_count, phase)
```
(5.9)

compares the state produced by QPE when using the manual inverse QFT with the state produced when using Qiskit's library QFT as the inverse transform. It constructs both circuits without measurement, converts them to statevectors, and uses `state_fidelity` to compute their overlap. For the tested parameters, this overlap is observed to be numerically close to 1, indicating that the manual inverse QFT behaves consistently with the built-in QFT in the QPE context.

5.3 Noise model and error analysis

5.3.1 Depolarizing + readout noise for QPE

Chapter 5 reuses the depolarizing-plus-readout noise model introduced in chapter 4, now applied to QPE experiments. The function

`build_simple_noise_model(cfg)` (5.10)

constructs a `NoiseModel` that:

- applies a one-qubit depolarizing error with probability `p1` to basic single-qubit gates (e.g. `"x"`, `"h"`, `"z"`, `"sx"`, `"rz"`, and, where supported, `"cp"` and `"swap"`);
- applies a two-qubit depolarizing error with probability `p2` to `"cx"` gates;
- adds a symmetric readout error with probability `p_meas` on measurement operations.

This construction matches the approach described in Qiskit Aer's noise-model building guide, where depolarizing channels and readout error matrices are attached to gates and measurements to simulate generic noise processes.

Using this noise model, QPE is run both in ideal and noisy modes. The resulting phase errors and dominant-outcome probabilities are plotted by:

- `plot_qpe_errors`, which shows absolute phase error for ideal vs noisy runs;
- `plot_qpe_success`, which shows the probability of the dominant outcome in both settings.

These plots make visible the trade-off between circuit depth and noise resilience in phase estimation: as noise is introduced, phase errors grow and the peak success probability declines, even for simple unitaries and eigenstates.

5.4 Grover refinement and capstone alignment

5.4.1 Grover iteration trade-off near the optimum

Although chapter 5 is primarily about QFT and QPE, the PoA explicitly states that Grover-based experiments should be refined to keep the capstone on track. To that end, the script includes a small helper

`grover_iteration_tradeoff_table(n_qubits, n_targets, max_extra)`, (5.11)

which computes the theoretical success probability

$$P_{\text{Grover}}(k) = \sin^2\big((2k+1)\theta\big), \qquad \sin^2\theta = \frac{M}{N},\ N = 2^n,\ M = \texttt{n_targets}, \tag{5.12}$$

for a range of iteration counts k near the optimal value

$$k_{\text{opt}} \approx \left\lfloor \frac{\pi}{4}\sqrt{\frac{N}{M}} \right\rfloor, \tag{5.13}$$

as discussed in the chapter 3 appendix. For each k, it records:

- the success probability $P_{\text{Grover}}(k)$;
- the distance $k - k_{\text{opt}}$.

The resulting table is printed and can also be plotted via `plot_grover_tradeoff`. This simple utility keeps the Grover work conceptually connected to the new QFT/QPE work by highlighting how sensitive the algorithm is to iteration count near the optimum, especially in scenarios where noise might shift the experimentally optimal k away from the textbook value [1].

5.4.2 Role in the Noise-aware Grover capstone

The Grover iteration trade-off table plays a supportive role for the Noise-aware Grover capstone:

- It provides a clean analytic baseline for success probability as a function of iteration count, independent of noise.
- When combined with the noise models and iteration sweeps from chapter 4, it allows direct comparison between ideal theoretical curves and noisy empirical curves.
- It emphasises that choosing k in noisy conditions is itself a non-trivial optimisation problem, which is one of the central themes of the planned capstone work on noise-aware Grover strategies.

In this sense, the chapter 5 Grover refinement is light in code but heavy in conceptual importance: it ensures that the QFT/QPE work does not drift away from the core capstone narrative.

Chapter 6

Noise-aware Grover Capstone

6.1 Grover implementation recap

6.1.1 Oracle and diffuser for single and multiple targets

We retain the hand-built oracle and diffuser structure developed earlier and formalise them as reusable circuit-building functions:

$$\texttt{build_oracle}(n,\ \texttt{targets}) \to U_f, \tag{6.1}$$

$$\texttt{build_diffuser}(n) \to D. \tag{6.2}$$

The oracle U_f implements the phase oracle

$$U_f \left|x\right\rangle = \begin{cases} -\left|x\right\rangle, & x \in \texttt{targets}, \\ \left|x\right\rangle, & \text{otherwise}, \end{cases} \tag{6.3}$$

for a set of marked basis states. In the code, each target integer is converted to an n-bit string. For each target:

1. Apply X gates to qubits where the corresponding bit is 0, mapping the selected state to $\left|11\cdots1\right\rangle$.
2. For $n = 1$, apply a single Z to flip the phase of the $\left|1\right\rangle$ state; for $n > 1$, perform a multi-controlled Z via the standard decomposition $H \cdot \mathrm{MCX} \cdot H$ on the last qubit.
3. Uncompute the initial X gates to restore the computational basis.

This procedure is repeated for each marked state, so the overall oracle applies a phase flip on all targets. The implementation is consistent with standard Grover oracle constructions found in Qiskit tutorials and IBM learning material.

The diffuser D implements inversion about the mean:

$$D = 2\,|s\rangle\,\langle s| - I, \qquad |s\rangle = H^{\otimes n}\,|0\rangle^{\otimes n}, \tag{6.4}$$

using the usual pattern:

1. apply $H^{\otimes n}$ followed by $X^{\otimes n}$;
2. apply a phase flip on $|0\rangle^{\otimes n}$, realised as $H \cdot \mathrm{MCX} \cdot H$ on the last qubit for $n > 1$, or a single Z for $n = 1$;
3. apply $X^{\otimes n}$ and then $H^{\otimes n}$ again.

This corresponds exactly to the standard diffuser described in many textbook treatments of Grover's algorithm [1].

6.1.2 Full Grover circuit and theoretical baseline

The full Grover circuit

$$\texttt{build_grover_circuit}(n,\ \texttt{targets},\ k,\ \texttt{measure},\ \texttt{add_barriers}) \tag{6.5}$$

is constructed as:

1. prepare the uniform superposition $|s\rangle = H^{\otimes n}\,|0\rangle^{\otimes n}$;
2. apply the oracle U_f and diffuser D in sequence, k times;
3. optionally measure the computational basis at the end.

The theoretical success probability for a single Grover run with k iterations, n qubits, and $M = |\texttt{targets}|$ marked states is given by the standard amplitude amplification formula:

$$P_{\mathrm{Grover}}(k) = \sin^2\big((2k+1)\theta\big), \qquad \sin^2\theta = \frac{M}{2^n}. \tag{6.6}$$

The script uses helper functions

$$\texttt{grover_angle}(n, M) \to \theta, \tag{6.7}$$

$$\texttt{theoretical_success_probability}(n, M, k) \to P_{\mathrm{Grover}}(k), \tag{6.8}$$

and the usual approximation

$$k_{\mathrm{opt}} \approx \left\lfloor \frac{\pi}{4\theta} \right\rfloor, \tag{6.9}$$

as the starting point for choosing iteration ranges. This matches the analytic baseline used in earlier chapters and the standard advice in IBM's Grover documentation.

6.2 Noise model design

6.2.1 Depolarizing and readout errors

To make Grover "noise-aware," we introduce a simple but widely-used noise model based on Qiskit Aer's `NoiseModel` API:

$$\texttt{build_simple_noise_model}(\texttt{NoiseConfig}), \tag{6.10}$$

where `NoiseConfig` packages three scalar parameters:

- `p1`: single-qubit depolarizing probability,
- `p2`: two-qubit depolarizing probability,
- `p_meas`: symmetric readout error probability.

The model is constructed as follows:

1. a one-qubit depolarizing channel with probability `p1` is attached to single-qubit gates such as ẍ, ḧ, ż, ṙz, and s̈x;
2. a two-qubit depolarizing channel with probability `p2` is attached to c̈x gates;
3. a symmetric readout error with probability `p_meas` is attached to all qubit measurements, via a simple 2×2 confusion matrix.

This style of noise model is exactly the pattern showcased in Qiskit Aer's noise-model tutorials, where `depolarizing_error` and `ReadoutError` are combined into a `NoiseModel` to simulate noisy circuits.

In the baseline experiments, we use a configuration such as

$$\texttt{p1} = 0.001, \quad \texttt{p2} = 0.01, \quad \texttt{p_meas} = 0.01, \tag{6.11}$$

which is intentionally modest but sufficient to observe noticeable degradation as the circuit depth grows.

6.2.2 Ideal vs noisy simulators

For each experiment, we run both an ideal and a noisy simulation:

- the ideal case uses an `AerSimulator` with no noise model attached;
- the noisy case uses an `AerSimulator` constructed with `noise_model=build_simple_noise_model(`

This mirrors Qiskit's recommended workflow for comparing noiseless and noisy circuit behaviour. All circuits are transpiled with a modest optimisation level (`optimization_level=1`) before execution.

6.3 Iteration sweeps and degradation analysis

6.3.1 Iteration sweep protocol

For each experiment specification

$$\texttt{ExperimentSpec(name, n_qubits, targets, shots)}, \tag{6.12}$$

we perform an iteration sweep using

$$\texttt{iteration_sweep(spec, max_iters, noise_cfg)}, \tag{6.13}$$

which, for each $k = 0, 1, \ldots, \texttt{max_iters}$, computes:

- the theoretical success $P_{\text{Grover}}(k)$,
- the exact success probability from a statevector simulation (no measurement, summing probabilities over target states),
- the sampled success probability from a shot-based simulation (ideal or noisy),
- circuit metrics such as depth and `cx` count after transpilation.

Shot-based success is defined as

$$P_{\text{sampled}}(k) = \frac{1}{\texttt{shots}} \sum_{x \in \texttt{targets}} \texttt{counts}[x], \tag{6.14}$$

where `counts` is the dictionary of measurement outcomes and each x is interpreted as its corresponding bitstring. The results for all k are stored as rows in a CSV file for later analysis and plotting.

The script uses three default experiment configurations:

- **Single Target n4**: $n = 4$ qubits, one target index (e.g. 11);
- **Multi Target n4**: $n = 4$ qubits, two targets (e.g. 3 and 11);
- **Single Target n5**: $n = 5$ qubits, one target index (e.g. 6).

These examples keep the Hilbert space small enough for statevector analysis while still illustrating multi-target behaviour and the impact of adding more qubits.

6.3.2 Ideal vs noisy success curves

For each experiment, the script generates a plot showing:

- the theoretical success curve $P_{\text{Grover}}(k)$;
- the ideal sampled success curve $P_{\text{ideal}}(k)$ from the noiseless simulator;
- the noisy sampled success curve $P_{\text{noisy}}(k)$ under the depolarizing-plus-readout noise model.

In the ideal case, $P_{\text{ideal}}(k)$ closely tracks the theoretical sinusoidal oscillation, with a maximum near the standard k_{opt}. In the noisy case, two main effects are observed:

1. the peak success probability is reduced relative to the ideal peak;
2. as k increases beyond the ideal optimum, the success probability declines more rapidly than in the ideal case, reflecting the accumulation of noise across repeated oracle and diffuser applications.

These observations align with the general understanding that Grover's algorithm becomes impractical on current noisy hardware beyond small problem sizes, as the depth increases and noise accumulates.

6.3.3 Degradation summary metrics

To quantify the degradation, the script computes a summary table:

$$\texttt{summarize_degradation(ideal_rows, noisy_rows)}, \tag{6.15}$$

which, for each iteration k, records:

- the ideal sampled success $P_{\text{ideal}}(k)$,
- the noisy sampled success $P_{\text{noisy}}(k)$,
- the absolute drop $\Delta P = P_{\text{ideal}}(k) - P_{\text{noisy}}(k)$,
- the relative drop percentage $\Delta P / P_{\text{ideal}}(k)$.

Plots of ΔP vs k provide a clear visual of how noise eats into Grover performance as iterations increase. Additionally, for each experiment, the script identifies:

- **best ideal iteration**: k that maximises $P_{\text{ideal}}(k)$;
- **best noisy iteration**: k that maximises $P_{\text{noisy}}(k)$.

In many runs, these best iterations differ, illustrating the key capstone point that the iteration count optimal in the ideal model may not be optimal on noisy hardware.

6.4 Noise-strength sweeps and depth analysis

6.4.1 Noise-strength sweeps

Beyond a fixed noise configuration, the script performs noise-strength sweeps using

$$\texttt{noise_strength_sweep(spec, noise_levels, fixed_iteration)}. \tag{6.16}$$

Here `noise_levels` is a list of two-qubit depolarizing probabilities p_2, e.g. $0.0, 0.002, 0.005, 0.01, 0.02, 0.04$
For each p_2, a configuration

$$\texttt{p1} = 0.1p_2, \quad \texttt{p2} = p_2, \quad \texttt{p_meas} \approx 0.5p_2 \tag{6.17}$$

is used to build the noise model. Two sweeps are run:

- at the ideal iteration k_{opt},
- at the best noisy iteration $k_{\text{best,noisy}}$ found from the baseline noisy sweep.

Plotting success probability vs p_2 for both cases illustrates how rapidly Grover performance decays as noise strength increases, and whether adjusting k can partially mitigate that decay. These noise sweeps align with studies in the literature that examine Grover under depolarizing or related noise channels.

6.4.2 Depth and CX-count correlation

To link performance degradation to circuit complexity, the script uses

$$\texttt{circuit_metrics}(n,\ \texttt{targets},\ k), \tag{6.18}$$

which:

- transpiles the Grover circuit to a basis such as $\{X, H, Z, RZ, SX, CX\}$;
- records total depth, number of two-qubit `CX` gates, and number of single-qubit gates.

Scatter plots of noisy success probability vs circuit depth (with the iteration index k encoded as colour) visually confirm the intuition that deeper circuits with more two-qubit gates are more heavily affected by the depolarizing and readout noise model. This again matches general findings in the literature and in Qiskit tutorials on noise-aware algorithm design.

6.5 Polished demonstration and capstone outcome

6.5.1 Polished demo structure

The `main` function orchestrates a polished capstone demo by:

1. printing an overview header describing the noise model and base configuration;
2. for each experiment (`Single Target n4`, `Multi Target n4`, `Single Target n5`), running a `polished_demo` that:
    - prints $N = 2^n$, the targets in both integer and bitstring form, and the theoretical k_{opt};
    - runs ideal and noisy iteration sweeps and prints an iteration-by-iteration table;
    - identifies and prints the best ideal and best noisy iteration and their success probabilities;
    - prints top measurement outcomes at those best iterations to highlight how probability mass shifts away from the target state under noise;
    - saves CSVs and plots for further analysis and inclusion in the report.
3. printing a brief "Capstone Outcome" summary with the main qualitative conclusions.

This yields both a narrative and a quantitative demonstration that can be reused for presentations or as an appendix in the final report.

6.5.2 Key capstone conclusions

The chapter 6 Noise-aware Grover capstone leads to several key conclusions:

- **Ideal vs theoretical alignment**: in the absence of noise, the sampled success probability closely follows the theoretical sinusoidal behaviour of Grover's algorithm, peaking near the analytic k_{opt}.
- **Degradation under noise**: under the depolarizing-plus-readout noise model, the peak success probability is significantly reduced and the success curve flattens as k increases, demonstrating how noise rapidly erodes Grover's advantage for deeper circuits.
- **Shifted optimal iteration**: the iteration that maximises success in the noisy case can differ from k_{opt}, confirming that noise-aware iteration strategies may be beneficial and that the textbook iteration count is not necessarily optimal on noisy hardware.

- **Noise-strength sensitivity**: increasing the depolarizing probabilities further suppresses success, even for small n, and the benefits of adjusting k diminish as noise grows, aligning with studies that show Grover's fragility under realistic noise channels.

- **Depth correlation**: the degradation correlates strongly with circuit depth and two-qubit gate count, reinforcing the view that reducing depth or error rates is crucial for practical Grover implementations in the NISQ era.

Overall, chapter 6 integrates the algorithmic and noise-model work from previous chapters into a coherent capstone: a Noise-aware Grover implementation that quantifies how and why Grover's speedup degrades under simple, tunable noise, supported by reproducible code and presentation-ready outputs.

Chapter 7

Quantum Sensing Applications in Astronomy, Navigation, and Materials Science

7.1 Quantum Sensing: Physical Foundations

7.1.1 Fragility as a Sensing Resource

Quantum sensing leverages the intrinsic fragility and coherence of quantum systems as a resource. A quantum sensor is typically a well-controlled quantum state - photons, trapped ions, cold atoms, or solid-state defects - whose phase, frequency, or population responds extremely sensitively to external fields or forces [13]. When this response is calibrated, small perturbations that would be undetectable by classical probes become measurable. Q-CTRL and other groups emphasise that the same fragility that is a liability in quantum computing can be turned into an advantage in sensing, using techniques such as dynamical decoupling and control "ruggedization" to suppress unwanted noise while retaining sensitivity to the signal [13, 14].

The key insight is that quantum coherence, when properly harnessed, enables the encoding of physical parameters into the relative phases of a superposition state. A generic quantum sensor operates by preparing a probe state ρ, allowing it to interact with the parameter θ of interest via a unitary $U(\theta) = e^{-iH\theta}$, and then performing a measurement whose outcome statistics reveal θ. The precision is fundamentally bounded by the quantum Cramér–Rao inequality: $\Delta\theta \geq 1/\sqrt{\nu F_Q}$, where ν is the number of measurements and F_Q is the quantum Fisher information. For separable (classical) states, $F_Q \leq N$ (standard quantum limit, SQL), while entangled states can achieve $F_Q \leq N^2$ (Heisenberg limit), offering a quadratic enhancement in scaling [29].

7.1.2 Interference, Superposition, and Entanglement

Three core mechanisms underlie most quantum sensing platforms:

- **Interference**: Quantum states in different paths of an interferometer pick up relative phases from external fields (gravity, rotation, refractive index variations), which are converted into measurable intensity or population differences. The phase shift $\Delta\phi$ is proportional to the interaction Hamiltonian and the interrogation time T.
- **Superposition**: Coherent superpositions of eigenstates allow small phase shifts to accumulate over time, enhancing sensitivity compared to classical mixtures. In an atom interferometer, for example, a two-photon Raman pulse creates a superposition of momentum states $|p\rangle$ and $|p + \hbar k_{\text{eff}}\rangle$; gravitational acceleration g imprints a phase $\Delta\phi = k_{\text{eff}} g T^2$.
- **Entanglement**: Entangled probes (e.g. squeezed states, GHZ states, or entangled ions) can saturate or approach Heisenberg-limited scaling, providing sensitivity beyond the standard quantum limit for tasks such as phase or frequency estimation. Squeezed vacuum states, for instance, reduce the variance in one quadrature below the vacuum level at the expense of increased variance in the conjugate quadrature.

These are the same physical concepts used in the quantum algorithms studied earlier - phase kickback in Deutsch and QPE, interference in Grover - now repurposed for metrology and sensing applications [13].

7.1.3 Noise, Decoherence, and Quantum Control

A critical challenge in quantum sensing is distinguishing the signal from environmental noise. The sensitivity of a quantum sensor is often characterised by its noise spectral density $S(f)$, which determines the minimum detectable signal at a given frequency. Dynamical decoupling (DD) sequences - periodic π-pulses that refocus dephasing noise - can extend coherence times T_2 by orders of magnitude while preserving sensitivity to signals at specific frequencies [30]. More sophisticated control techniques, such as composite pulses, shaped pulses, and quantum error mitigation, further tailor the sensor's frequency response, enabling narrowband or broadband operation as required by the application.

7.2 Astronomical Interferometry and Quantum Enhancement

7.2.1 Classical Astronomical Interferometry

Classical astronomical interferometry combines light collected by separated telescopes to synthesise a larger effective aperture, thereby improving angular resolution beyond what a single mirror can provide. The angular resolution of an interferometer with baseline B observing at wavelength λ is approximately $\theta \sim \lambda/B$. For optical wavelengths and kilometre-scale baselines, this reaches microarcsecond precision - sufficient to resolve stellar surfaces, accretion disks, and exoplanetary systems.

Optical and infrared interferometers measure fringe visibility $V = (I_{\max} - I_{\min})/(I_{\max} + I_{\min})$ and phase to reconstruct images of astrophysical structures. However, losses, atmospheric turbulence, and limited coherence across long baselines constrain sensitivity and dynamic range [15]. Photon loss is particularly severe: for a baseline B, the field attenuation scales as $e^{-\alpha B}$, where α is the atmospheric or waveguide loss coefficient. This exponential degradation fundamentally limits the longest usable baselines for classical interferometry.

7.2.2 Quantum-Assisted Optical Interferometry

Quantum information concepts have recently been brought to bear on astronomical interferometry. Several proposals and experiments explore quantum-assisted telescope arrays that circumvent classical limitations:

- **Quantum-assisted interferometry via networks**: Schemes have been proposed where incoming stellar photons are combined with entangled resources distributed across a telescope network, enabling non-local measurements that circumvent exponential field attenuation with baseline length and loss [16,17]. The core idea is that entanglement can teleport phase information between stations without physically transmitting the astronomical signal itself, thereby avoiding loss.

- **Continuous-variable (CV) schemes**: Work on continuous-variable quantum teleportation and two-mode squeezed states suggests ways to use CV entanglement to share phase information between distant telescopes while minimising photon loss and avoiding the need to store or clone the astronomical signal itself [18]. Two-mode squeezed vacuum states, generated via parametric down-conversion, exhibit Einstein–Podolsky–Rosen (EPR) correlations that can be harnessed for phase-sensitive measurements.

- **Entanglement-assisted non-local interferometry**: Recent experiments demonstrate entanglement-assisted optical interferometry using quantum memories, providing a route to optimal non-local measurements for faint sources across long baselines, and showing that quantum correlations can help reach fundamental sensitivity bounds in telescope arrays [17]. In these demonstrations, heralded single photons from a down-conversion source are stored in quantum memories at two stations, enabling time-delayed interference that mimics a single large-aperture telescope.

These developments connect directly to the interference and phase-kickback concepts in earlier chapters: instead of encoding a computational answer in phase, the phase encodes astrophysical information (e.g. source structure, angular diameter, or binary orbital parameters) and entanglement helps collect that information more efficiently across noisy, lossy channels. The quantum advantage here is not merely one of sensitivity but of *information efficiency*: entanglement-assisted protocols can, in principle, recover visibility information that would be completely lost to photon statistics in classical schemes.

7.2.3 Squeezed Light and Gravitational-Wave Astronomy

A mature and impactful application of quantum-enhanced interferometry is gravitational-wave detection. Ground-based detectors such as LIGO, Virgo, and KAGRA are kilometre-scale Michelson interferometers whose sensitivity is ultimately limited by quantum noise: shot noise at high frequencies and radiation-pressure noise at low frequencies. The injection of squeezed vacuum into the dark port of the interferometer reduces the uncertainty in the phase quadrature, directly improving the signal-to-noise ratio for gravitational-wave strain h.

Frequency-dependent squeezing, now operational in LIGO during its fourth observing run (O4), achieves sub-standard-quantum-limit performance by rotating the squeezing angle as a function of frequency, simultaneously addressing both shot noise and radiation-pressure noise regimes [37]. For ground-based detectors operating in the shot-noise-dominated regime, squeezed vacuum provides a sensitivity enhancement factor $\mathcal{E} \approx 1.8$–2.4, corresponding to a factor of 3–6 increase in detection rate for isotropically distributed sources. The space-based detector LISA, approved by ESA for launch in the 2030s, operates in a fundamentally different noise regime where classical noise sources dominate; quantum enhancement is therefore marginal ($\mathcal{E} \approx 1.04$) but still valuable for the most extreme sources [31].

7.2.4 Atom Interferometry for Gravitational-Wave Detection

Atom interferometric detectors target the mid-band frequency gap of $\sim$0.01–10 Hz, lying between the LISA band ($\lesssim 0.1$ Hz) and the LIGO band ($\gtrsim 10$ Hz) [31]. This gap is sci-

entifically rich: it contains the early inspiral of stellar-mass binary black holes (hours to days before merger), intermediate-mass black hole mergers (10^2–$10^4\,M_\odot$), and multiband gravitational-wave astronomy events. Because atoms are in free fall during the interrogation period, with no mechanical connection to the ground, atom interferometers are immune to suspension thermal noise and partially immune to seismic noise - the dominant limitations for ground-based optical detectors at low frequencies.

The fundamental coupling mechanism for gravitational-wave detection is light propagation over macroscopic baselines, which provides an enormous transducer gain compared to internal atomic coupling (tidal distortion of electronic wavefunctions) or centre-of-mass coupling (Doppler shifts from geodesic motion). The transducer gain for internal atomic coupling is $G_A \sim 10^{-20}$, yielding a deficit of $\sim 10^{35}$ relative to laser interferometry - an insurmountable gap that exceeds any projected quantum enhancement [31]. This analysis underscores a crucial principle: the viability of a quantum sensing scheme depends not only on the sensor's quantum performance but on the physical coupling mechanism between the signal and the observable.

7.3 Matter-Wave Interferometry and Space-Science Sensing

7.3.1 Atom Interferometers as Inertial and Gravity Sensors

Cold-atom interferometers use matter waves rather than light. Laser pulses split and recombine atomic wave packets, and gravitational fields, inertial forces, and rotations imprint phase shifts on the different paths. When the packets are recombined, the resulting interference pattern encodes acceleration, gravity, and rotation information [13, 19–21]. The phase shift in a three-pulse Mach–Zehnder atom interferometer under gravitational acceleration is

$$\Delta\phi = k_{\text{eff}}\, g\, T^2, \tag{7.1}$$

where k_{eff} is the effective wave number of the Raman or Bragg transition and T is the interrogation time between pulses. In microgravity, T can be extended from milliseconds (terrestrial) to tens of seconds, yielding dramatic sensitivity improvements.

These atom interferometers act as:

- **High-precision accelerometers and gyroscopes** for inertial navigation, with demonstrated sensitivities of $3 \times 10^{-9}\, g/\sqrt{\text{Hz}}$ at 100 ms measurement periods and per-shot sensitivities as low as $6.7 \times 10^{-12}\, g$ in laboratory demonstrations [33];

- **Gravimeters and gravity gradiometers** for Earth and planetary science, capable of resolving gravity variations at the part-per-billion level;

- **Platforms for fundamental physics tests**, including tests of the equivalence principle, searches for ultralight dark matter, and constraints on dark energy models [21].

Their sensitivity arises from long coherent interrogation times, precise laser control, and the direct coupling of atomic phase to inertial and gravitational effects [21].

7.3.2 Space-Based Atom Interferometry

Deploying atom interferometers in space further enhances their capabilities. NASA's Cold Atom Lab (CAL) aboard the ISS and related experiments explore matter-wave interferometry in microgravity, where atoms can be kept in free fall for long durations without falling out of the apparatus [19, 35]. The CAL Science Team has published milestone papers documenting the first sustained atom interferometry in space, demonstrating that these exquisitely sensitive instruments can be operated remotely from Earth - a critical proof of concept for future deep-space missions [19, 35].

Longer interrogation times increase phase accumulation and thus sensitivity:

- **Gravity mapping and geodesy**: Atom interferometers can measure small variations in gravitational fields and gradients, enabling detailed gravity maps for Earth and potentially other planetary bodies. This supports the study of ice sheets, aquifers, and subsurface structures. Future satellite gravity missions may combine cold-atom interferometry (CAI) accelerometers with classical electrostatic accelerometers to exploit complementary noise characteristics across the measurement band, with CAI offering superior long-term stability for low-frequency gravity signals [33, 36].

- **Navigation and PNT**: Quantum inertial sensors provide absolute, drift-free measurements of acceleration and rotation, offering improved positioning and navigation in GPS-denied or deep-space environments [22–24]. Hybrid quantum-classical architectures - combining cold-atom interferometers with fibre-optic gyroscopes and mechanical accelerometers - have demonstrated operation over wide angle ranges (up to 30°) and high rotation rates (up to 14°/s), meeting operational requirements for undersea, sea, air, and space applications [34].

- **Fundamental physics**: Long-baseline space interferometers can probe tiny deviations from general relativity, search for new forces, or constrain models of dark matter and dark energy [21].

7.3.3 China's Space-Based Cold Atom Gyroscope

A landmark achievement in 2025 was the demonstration of the world's first cold atom gyroscope operating in space aboard the China Space Station. The China Space Station Atom Interferometer (CSSAI), launched in late 2022, uses rubidium atoms cooled to near absolute zero to detect rotational changes as small as the slow spin of a coin observed from over 100 kilometres away. Its acceleration resolution is approximately 100,000 times more sensitive than a typical smartphone accelerometer [32].

A key technical challenge solved by the CSSAI team was "shearing fringe dephasing - distortions of interference patterns in orbit arising from small differences in the initial position or velocity of ultracold atom clouds. By identifying a "magic angle" laser beam configuration that cancels out unwanted shifts, the team achieved stable rotation and acceleration readings with a single-experiment rotation resolution of 50 μrad/s and an acceleration resolution of 1.0 μm/s^2. Over longer integration periods, the rotation resolution improved to 17 μrad/s, closely matching conventional gyroscopes aboard the station [32].

This demonstration is strategically significant: it paves the way for quantum gyroscopes to replace or complement existing systems in satellites, spacecraft, submarines, and other vehicles where traditional sensors reach their limits. It also opens new paths for testing subtle predictions of general relativity - such as frame-dragging - with quantum precision.

For navigation-related use cases specifically, quantum inertial units can be combined with gravity maps and magnetic field models to enable gravity-aided and field-matching navigation with reduced drift and high accuracy [22–24].

7.4 Quantum Magnetometry and Navigation

7.4.1 Diamond NV Magnetometers

Solid-state quantum sensors based on nitrogen-vacancy (NV) centres in diamond are among the most mature nanoscale magnetometry platforms. NV centres are atomic-scale defects whose spin state can be read out optically and manipulated with microwaves; their Zeeman shifts provide high-sensitivity measurements of local magnetic fields over a wide range of temperatures and bandwidths [25, 26].

The NV centre is a point defect in the diamond lattice consisting of a substitutional nitrogen atom adjacent to a vacancy. Its electronic ground state is a spin triplet ($S = 1$) with a zero-field splitting $D \approx 2.87$ GHz between the $m_s = 0$ and $m_s = \pm 1$ sublevels. In the presence of a magnetic field B, the $m_s = \pm 1$ levels split by $\Delta E = g\mu_B B$, where $g \approx 2$ is the Landé g-factor and μ_B is the Bohr magneton. By driving optically detected

magnetic resonance (ODMR) transitions and monitoring fluorescence intensity, magnetic fields can be measured with sensitivities approaching ~ 10 aT·cm$^{3/2}/\sqrt{\text{Hz}}$ at geomagnetic fields.

These sensors are:

- **Quantitative and non-invasive**: They measure magnetic fields without perturbing the sample, enabling in-situ characterisation of living biological systems and delicate materials.
- **Robust and operable from cryogenic to room temperatures**: Unlike superconducting quantum interference devices (SQUIDs), NV sensors require no cryogenics, making them deployable in field settings.
- **Capable of nanoscale spatial resolution and noise spectroscopy**: By localising NV centres near sample surfaces (e.g. via ion implantation or nitrogen delta-doping), spatial resolution below 10 nm can be achieved, sufficient to image single electron spins and magnetic domains.

In materials science, NV magnetometry has been used to study current flow, spin order, and magnetic dynamics in condensed-matter systems, as discussed further [25, 26].

7.4.2 Magnetic-Field-Based Navigation

Quantum magnetometers can also contribute to navigation:

- By measuring fine structure in Earth's magnetic field and matching it to geomagnetic maps, vehicles can determine position without relying on GPS, enabling magnetic field-matching navigation [23, 24]. The International Geomagnetic Reference Field (IGRF) provides a global model, but local anomalies - which vary on scales of kilometres to metres - are the primary navigation signal.
- Combined with atom interferometer-based gravimeters and accelerometers, quantum magnetometry forms part of hybrid quantum-classical navigation architectures that reduce drift and achieve sub-metre accuracy over long durations [23].

7.5 Quantum Sensing in Materials Science

7.5.1 NV-Based Nanoscale Sensing of Condensed Matter

In materials science and condensed-matter physics, quantum sensors based on NV centres in diamond have become powerful tools for probing magnetic and electronic properties at the nanoscale. Technical reviews emphasise several key features: quantitative field

measurement, non-invasive operation, robustness, nanoscale resolution, and broad temperature compatibility [25]. These sensors have been used to:

- **Image static magnetic fields** arising from spin order and current distributions in thin films, heterostructures, and superconducting devices. By raster-scanning an NV centre across a sample surface, two-dimensional magnetic field maps can be reconstructed with spatial resolution determined by the NV–sample separation.
- **Perform noise spectroscopy** of fluctuating electrical currents and magnetic dynamics, accessing correlation functions and order parameters that are difficult to probe with traditional techniques. Dynamical decoupling sequences (e.g. Carr–Purcell–Meiboom–Gill, XY8) tailor the sensor's spectral response to specific noise frequencies, enabling the decomposition of magnetic noise into its constituent spectral components.
- **Study many-body physics phenomena**, including spin textures, topological states, and correlated electron systems. NV magnetometry has revealed the magnetic structure of skyrmions - topologically protected spin textures - in chiral magnets and the Meissner screening currents in superconductors.

The same quantum control tools used for error mitigation in computation (e.g. dynamical decoupling) can be used in NV sensing to tailor the frequency response of the sensor to different noise spectra [25]. This convergence of quantum control techniques across computing and sensing is a recurring theme in the broader quantum technology landscape.

7.5.2 Other Sensing Platforms

Beyond NV centres, quantum sensing in materials science includes:

- **Trapped ions** used as force and electric-field sensors at the microscale, with demonstrated sensitivities to electric fields at the μV/m level and forces at the yoctonewton (10^{-24} N) scale.
- **Superconducting circuits and SQUIDs** for ultra-low-field magnetometry and current sensing, operating at cryogenic temperatures but offering unmatched sensitivity for macroscopic samples.
- **Optomechanical and spin-based sensors** for stress, strain, and temperature at high precision, leveraging the coupling between mechanical motion and optical or spin degrees of freedom.

These platforms complement NV sensors and together provide a rich landscape of quantum-enhanced probes for material properties, interfaces, and defects [21]. For the report, the

focus remains on NV-based sensing as a representative example with strong existing literature and clear connections to both fundamental physics and potential industrial applications.

7.6 Long-Term Exploratory Angle: Asteroid-Mining-Related Space Applications

7.6.1 Asteroid Resource Prospecting

Asteroid mining remains a long-term, high-uncertainty topic, but resource prospecting is already an active area of research. Reviews of asteroid resource prospecting describe:

- **Robotic missions** (including CubeSats and small landers) equipped with spectrometers, imagers, and sampling systems to survey near-Earth asteroids and characterise their composition. Missions such as NASA's OSIRIS-REx and JAXA's Hayabusa2 have already returned samples from carbonaceous asteroids, demonstrating the feasibility of close-proximity operations.
- **Mapping of water, metals, and other resources** relevant for propellant, life support, and construction materials. C-type asteroids, comprising about three-quarters of known asteroids, contain water-bearing silicate and carbon-based minerals; S-type asteroids are richer in high-temperature metals.
- **Creation of resource catalogues** that inform future mining operations and space resource utilisation strategies [27, 28].

Although current prospecting techniques are mostly classical (spectroscopy, imaging, sampling), quantum-enhanced sensing tools - such as ultra-sensitive gravimetry or magnetometry - could in principle contribute to future missions by providing more precise measurements of local gravity fields, internal density distributions, and magnetic signatures.

7.6.2 Potential Quantum-Sensing Contributions

In a long-term scenario, quantum sensing might support asteroid mining and resource utilisation via:

- **Gravity mapping**: Space-deployed atom interferometers could measure subtle gravity gradients around small bodies, revealing internal structure, voids, and mass distribution, which are crucial for assessing mining feasibility and stability. The low gravity and irregular shapes of small bodies make classical gravimetry challenging;

quantum sensors' absolute reference frames and high sensitivity could overcome these limitations.

- **Magnetic and materials analysis**: Compact NV or other quantum magnetometers might be used to characterise magnetic properties and infer composition at close range, complementing spectroscopic data [21, 25]. Magnetic signatures can distinguish between metallic (M-type) and stony (S-type) compositions, aiding target selection.

- **Navigation and operations**: Quantum inertial and gravity sensors could provide robust navigation and station-keeping near irregular, low-gravity objects where conventional GNSS is unavailable and classical inertial sensors drift substantially [22–24]. The CSSAI demonstration aboard the China Space Station illustrates the trajectory toward such capabilities.

It is important to note that these ideas are speculative and depend on future maturation of quantum sensing platforms, miniaturisation, and integration with space-qualified hardware.

7.7 Consolidation and Comparative Insights

The study of quantum sensing applications leads to several comparative insights:

- **Common physical foundations**: Astronomical interferometry, atom interferometry for navigation, and NV-based materials sensing all rely on interference, phase accumulation, and precise quantum control, linking them conceptually to the algorithmic work on Deutsch, Grover, and QPE. The mathematical structures - superposition, unitary evolution, projective measurement - are identical; only the Hamiltonians and observables differ.

- **Different deployment constraints**: Astronomy and space-science applications face challenges of long baselines, transmission loss, and space qualification; navigation applications emphasise robustness, miniaturisation, and hybrid architectures; materials-science applications focus on nanoscale resolution and non-invasive probing in laboratory settings.

- **Navigation as a near-term domain**: Quantum inertial and magnetic sensors already demonstrate practical navigation improvements (reduced drift, GPS-denied operation, gravity-aided positioning) in terrestrial and near-Earth contexts. The 2026 review by [33] identifies sensor robustness, miniaturisation, seamless integration, and the development of protocols to quantify quantum performance as priorities for advancing field-deployable quantum positioning and navigation solutions.

- **Materials sensing as a physics driver**: NV-based quantum sensing is strongly tied to condensed-matter and many-body physics, providing unique access to correlation functions, order parameters, and nanoscale dynamics that are otherwise difficult to measure [25,26]. The ability to image single spins and measure nanotesla-scale fields at room temperature represents a paradigm shift in experimental condensed-matter physics.

- **Asteroid mining as a speculative frontier**: While asteroid resource prospecting is advancing, quantum sensing contributions are at present largely conceptual; they nonetheless illustrate how the tools studied in earlier chapters could eventually migrate from laboratory and Earth to small-body exploration [21, 27, 28].

- **Coupling mechanism as the determining factor**: The analysis of gravitational-wave detection [31] reveals that the viability of a quantum sensing scheme is determined not by the sensor's quantum performance alone but by the physical coupling mechanism between the signal and the observable. This principle - that transducer gain, not sensor precision, is the primary arbiter of applicability - generalises to all quantum sensing domains.

Appendix A

Implementation Notes: Refactored Deutsch's Algorithm Code

A.1 Overview of the code structure

This appendix documents the main design choices in the refactored Qiskit implementation of Deutsch's algorithm used in chapter 2. The goal is not to reproduce the full listing, but to explain the structure and the role of each layer of the code. The implementation follows the same high-level pattern as the IBM Quantum examples for Deutsch and Deutsch–Jozsa: construct an oracle U_f, embed it into the Deutsch circuit, analyze the statevector, and then perform shot-based simulation on a backend such as `AerSimulator` [5, 6].

The script is organized into four logical blocks:

1. a data structure describing the four allowed oracles;
2. an oracle factory that builds U_f for each f;
3. Deutsch-circuit construction, exact analysis, and classification utilities;
4. plotting and a main routine that compares all four oracles.

This modular design makes it easy to extend the implementation to Deutsch–Jozsa and related algorithms by changing only the oracle and the number of query qubits.

A.2 Oracle metadata and factory

The first part of the script defines a small data class to hold metadata about each oracle:

```
@dataclass(frozen=True)
class OracleSpec:
    key: str
```

```
    label: str
    formula: str
    oracle_type: str   # "CONSTANT" or "BALANCED"
```

and a dictionary

```
ORACLES = {
    "constant_0": OracleSpec("constant_0", "Constant-0", "f(x)=0", "CONSTANT"),
    "constant_1": OracleSpec("constant_1", "Constant-1", "f(x)=1", "CONSTANT"),
    "identity":   OracleSpec("identity",   "Identity",   "f(x)=x", "BALANCED"),
    "not":        OracleSpec("not",        "NOT",        "f(x)=1-x", "BALANCED"),
}
```

This allows the rest of the code to work with symbolic keys such as `"identity"` while still having access to a human-readable label, a compact formula for $f(x)$, and the expected classification (constant or balanced).

The actual oracle circuit is constructed by a factory function:

```
def make_oracle(oracle: str) -> QuantumCircuit:
    if oracle not in ORACLES:
        raise ValueError(...)
    uf = QuantumCircuit(2, name=f"U_f[{oracle}]")

    if oracle == "constant_0":
        pass
    elif oracle == "constant_1":
        uf.x(1)
    elif oracle == "identity":
        uf.cx(0, 1)
    elif oracle == "not":
        uf.x(0)
        uf.cx(0, 1)
        uf.x(0)
    return uf
```

The two-qubit convention is:

- qubit 0: query register (x);
- qubit 1: ancilla (y).

The four cases correspond directly to the functions discussed in the main text:

- `"constant_0"`: $f(x) = 0$, so U_f is the identity;

- `"constant_1"`: $f(x) = 1$, so the ancilla is flipped unconditionally by an X gate;
- `"identity"`: $f(x) = x$, so a CNOT from the query to the ancilla encodes $f(x)$ as a controlled flip;
- `"not"`: $f(x) = 1 - x$, implemented as an X on the query, then CNOT, then another X, which effectively gives a control on $|0\rangle$.

Writing the oracle as a separate `QuantumCircuit` object emphasizes that U_f is a reusable unitary that can be appended into different larger circuits, consistent with the way Qiskit tutorials treat query oracles [5, 6].

A.3 Deutsch circuit construction

The Deutsch circuit itself is built by

```
def build_deutsch_circuit(
    oracle: str,
    measure: bool = True,
    barriers: bool = True
) -> QuantumCircuit:
    qr = QuantumRegister(2, "q")
    cr = ClassicalRegister(1, "c") if measure else None
    qc = QuantumCircuit(qr, cr) if measure else QuantumCircuit(qr)

    # |0>|1>
    qc.x(qr[1])

    # |+>|->
    qc.h(qr[0])
    qc.h(qr[1])

    if barriers:
        qc.barrier()

    qc.append(make_oracle(oracle).to_gate(), [qr[0], qr[1]])

    if barriers:
        qc.barrier()

    qc.h(qr[0])
```

```
    if measure:
        qc.measure(qr[0], cr[0])
    return qc
```

Key points:

- The `measure` flag controls whether the circuit includes measurement and a classical register. This is convenient because exact statevector analysis requires a circuit without final measurements, while shot-based simulation and plotting require measurements.
- The `barriers` flag is used purely for readability in the text diagram produced by `qc.draw(output="text")`. Removing barriers does not change the unitary, but it can change the optimized circuit after transpilation.
- The sequence `x`, `h`, oracle, and final `h` matches the analytical description in the main chapter 2: prepare $|0\rangle|1\rangle$, transform to $|+\rangle|-\rangle$, apply U_f, and then apply the final Hadamard on the query qubit.

A.4 Exact analysis via Statevector

For the theoretical analysis and debugging, the code uses Qiskit's `Statevector` class from the `quantum_info` module, which allows one to compute exact pre-measurement states and derived probabilities without performing a projective measurement [10]. A helper function computes the ideal marginal distribution of the query qubit:

```
def exact_query_distribution(oracle: str) -> Tuple[Dict[str, float], Statevector]:
    qc = build_deutsch_circuit(oracle, measure=False, barriers=False)
    sv = Statevector.from_instruction(qc)
    probs_q0 = sv.probabilities_dict(qargs=[0])
    dist = {"0": float(probs_q0.get("0", 0.0)),
            "1": float(probs_q0.get("1", 0.0))}
    return dist, sv
```

The call to

```
Statevector.from_instruction(qc)
```

creates a `Statevector` object representing the final state of the circuit, and the method `probabilities_dict(qargs=[0])` returns a dictionary of exact probabilities for measuring the specified qubit in the computational basis. This is exactly the type of usage recommended in the IBM and Qiskit documentation for state-level analysis [10].

To classify an oracle from the exact distribution, a simple tolerance-based function is used:

```
def classify_from_distribution(dist: Dict[str, float], tol: float = 1e-9) -> str:
    if abs(dist["0"] - 1.0) < tol:
        return "CONSTANT"
    if abs(dist["1"] - 1.0) < tol:
        return "BALANCED"
    return "INDETERMINATE"
```

This reflects the fact that, in the ideal algorithm, the query qubit has deterministic measurement outcomes at the end of the circuit: $|0\rangle$ for constant and $|1\rangle$ for balanced oracles.

In addition to the final state, the implementation prints intermediate statevectors using a helper that reconstructs the circuit step by step and calls `Statevector.from_instruction` at four checkpoints: after initialization, after the Hadamard layer, after the oracle, and after the final Hadamard. This makes it easier to see how phases are introduced and then converted into amplitudes.

A.5 Shot-based simulation with AerSimulator

To mimic actual experimental runs and to prepare for later work with noisy devices, the same circuits are also simulated using `AerSimulator`, Qiskit's high-performance simulator backend [11]. The relevant function is:

```
def run_shot_simulation(oracle: str, shots: int = 1024) -> Dict[str, int]:
    qc = build_deutsch_circuit(oracle, measure=True, barriers=True)
    sim = AerSimulator()
    tqc = transpile(qc, sim)
    result = sim.run(tqc, shots=shots).result()
    counts = result.get_counts(tqc)
    return dict(counts)
```

The pattern here is standard for Qiskit:

1. Build a circuit that includes measurements.
2. Select a simulator backend (`AerSimulator` from `qiskit_aer`).
3. Transpile the circuit to the backend using `transpile`.
4. Run the circuit with a chosen number of shots.

5. Extract the counts via `get_counts`.

In the current implementation, only ideal simulation is used (no noise model), but the same backend supports noisy simulation via configuration options documented in the Aer guides [11]. This provides a natural path for future experiments where the impact of noise on Deutsch's algorithm is studied explicitly.

A simple majority rule is used to classify the oracle based on the shot counts:

```
def classify_from_counts(counts: Dict[str, int]) -> str:
    dominant = max(counts, key=counts.get)
    return "CONSTANT" if dominant == "0" else "BALANCED"
```

Comparing this classification with the exact classification from the statevector analysis demonstrates that the simulator reproduces the ideal theoretical behaviour in the large-shot limit.

A.6 Unified analysis and plotting

To organise the results, the script defines a function that runs both the exact and shot-based analyses and returns a single dictionary of outcomes:

```
def analyze_oracle(oracle: str, shots: int = 1024) -> dict:
    spec = ORACLES[oracle]

    exact_dist, sv = exact_query_distribution(oracle)
    exact_verdict = classify_from_distribution(exact_dist)

    counts = run_shot_simulation(oracle, shots=shots)
    shot_verdict = classify_from_counts(counts)

    return {
        "oracle": oracle,
        "label": spec.label,
        "formula": spec.formula,
        "expected": spec.oracle_type,
        "exact_distribution": exact_dist,
        "exact_verdict": exact_verdict,
        "counts": counts,
        "shot_verdict": shot_verdict,
        "statevector": sv,
    }
```

The plotting function then loops over all four oracles, calls `analyze_oracle`, and builds a row of bar charts showing the counts for outcomes 0 and 1 for each case. The plot titles display the function formula $f(x)$, and a text annotation in each subplot shows the classification derived from the counts (constant or balanced). This makes it visually obvious that:

- the two constant oracles concentrate counts on outcome 0;
- the two balanced oracles concentrate counts on outcome 1.

Appendix B

Grover Amplitudes and Code-Level Tracking

B.1 Analytic amplitude evolution in Grover's algorithm

For a search space of size $N = 2^n$ with M marked items, the standard analysis of Grover's algorithm proceeds by restricting attention to the two-dimensional subspace spanned by

$$|\omega\rangle = \frac{1}{\sqrt{M}} \sum_{x \text{ marked}} |x\rangle\,, \qquad |\mu\rangle = \frac{1}{\sqrt{N-M}} \sum_{x \text{ unmarked}} |x\rangle\,. \tag{B.1}$$

The initial uniform superposition can be written as

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle = \sqrt{\frac{M}{N}}\ |\omega\rangle + \sqrt{\frac{N-M}{N}}\ |\mu\rangle\,. \tag{B.2}$$

It is conventional to parameterize this decomposition by an angle θ defined via

$$\sin\theta = \sqrt{\frac{M}{N}}, \qquad \cos\theta = \sqrt{\frac{N-M}{N}}, \tag{B.3}$$

so that

$$|s\rangle = \sin\theta\ |\omega\rangle + \cos\theta\ |\mu\rangle\,. \tag{B.4}$$

One Grover iteration G consists of the oracle U_f, which flips the phase of $|\omega\rangle$, followed by the diffuser D, which performs a reflection about $|s\rangle$. Restricted to the span of $\{|\omega\rangle\,, |\mu\rangle\}$, the combined operation acts as a rotation by angle 2θ toward $|\omega\rangle$ [1]. After k Grover iterations, the state is

$$G^k\,|s\rangle = \sin\big((2k+1)\theta\big)\,|\omega\rangle + \cos\big((2k+1)\theta\big)\,|\mu\rangle\,, \tag{B.5}$$

so the probability of measuring any marked state is

$$P_{\text{marked}}(k) = \left|\sin\left((2k+1)\theta\right)\right|^2. \tag{B.6}$$

For the single-target case $M = 1$, $\sin\theta = 1/\sqrt{N}$ and the amplitude of the marked basis state grows approximately sinusoidally as a function of k. This analytic expression is exactly what the `track_amplitudes` function is numerically sampling via Qiskit's `Statevector` class.

B.2 Choosing the number of iterations

The optimal number of Grover iterations is chosen to make the marked-state probability in Eq. (B.6) close to one. We want

$$\sin^2\left((2k+1)\theta\right) \approx 1, \tag{B.7}$$

which is achieved when

$$(2k+1)\theta \approx \frac{\pi}{2}. \tag{B.8}$$

Solving for k gives

$$k \approx \frac{\pi}{4\theta} - \frac{1}{2}. \tag{B.9}$$

In terms of N and M,

$$\theta = \arcsin\sqrt{\frac{M}{N}}, \tag{B.10}$$

so

$$k \approx \frac{\pi}{4}\sqrt{\frac{N}{M}} - \frac{1}{2}. \tag{B.11}$$

Since k must be an integer, it is standard to take

$$k_{\text{opt}} = \left\lfloor \frac{\pi}{4\theta} \right\rfloor \approx \left\lfloor \frac{\pi}{4}\sqrt{\frac{N}{M}} \right\rfloor, \tag{B.12}$$

which is the same formula implemented in the code via

```
return max(1, round((np.pi / 4) * np.sqrt(N / n_targets)))
```

for $N = 2^{n_qubits}$ and `n_targets = M`. This choice matches the iteration-selection rule described in many expositions of Grover's algorithm, including IBM's learning material on "Choosing the number of iterations".

The analytic behaviour explains the numerical patterns observed in the chapter 3 plots:

- For small k, the marked-state probability grows approximately quadratically in k.

- Around k_{opt}, the probability peaks near one.
- For larger k, the probability oscillates and eventually decreases again, which is why "over-rotating" beyond the optimal iteration count reduces the success probability.

The `track_amplitudes` routine numerically recovers these oscillatory curves by repeatedly evolving a `Statevector` under the oracle and diffuser circuits and reading out the amplitude of the target basis state at each iteration.

B.3 Matching the analytic curve to the code

From a code perspective, the key observation is that the `track_amplitudes` function is effectively sampling Eq. (B.6) at integer values of k, using Qiskit's exact statevector simulator [10]. The steps are:

1. Build the uniform superposition state $|s\rangle = H^{\otimes n} |0\rangle^{\otimes n}$.
2. For each iteration $k = 0, 1, \ldots,$ `max_iters`:
   (a) use `Statevector.probabilities_dict()` to read out the probability $P_{\text{target}}(k)$ of the marked basis state;
   (b) apply one Grover step (oracle plus diffuser) to the statevector using `sv.evolve(step.decompose`

To make the analytic connection explicit in the report, one can:

- compute $\theta = \arcsin(\sqrt{M/N})$ and plot the theoretical curve

$$P_{\text{th}}(k) = \sin^2\big((2k+1)\theta\big)$$

  alongside the numerically extracted probabilities from `track_amplitudes`;
- check that the iteration k at which the numerical curve peaks matches the value produced by `optimal_iterations`.

This joint analytic–numerical view reinforces the interpretation of Grover's algorithm as a rotation in a two-dimensional subspace, and it confirms that the implementation is faithfully reproducing the textbook behaviour of amplitude amplification described in the literature and online tutorials [1].

Appendix C

QFT and QPE Analytic Notes

C.1 Explicit QFT matrices for small n

For n qubits, the quantum Fourier transform QFT_n is the unitary

$$\mathrm{QFT}_n |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i x k/2^n} |k\rangle , \qquad x \in \{0, \ldots, 2^n - 1\}. \tag{C.1}$$

For small n, it is useful to write QFT_n explicitly as a matrix.

C.1.1 One-qubit case ($n = 1$)

For $n = 1$, the QFT is just the Hadamard transform:

$$\mathrm{QFT}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{C.2}$$

This matches the implementation in the code, where the outermost call to `apply_qft_rotations` on a single qubit reduces to a Hadamard gate with no controlled-phase rotations.

C.1.2 Two-qubit case ($n = 2$)

For $n = 2$, we have $N = 4$ and basis states $\{|0\rangle , |1\rangle , |2\rangle , |3\rangle\}$. The QFT matrix is

$$\mathrm{QFT}_2 = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & e^{\frac{\pi i}{2}} & e^{\pi i} & e^{\frac{3\pi i}{2}} \\ 1 & e^{\pi i} & 1 & e^{\pi i} \\ 1 & e^{\frac{3\pi i}{2}} & e^{\pi i} & e^{\frac{\pi i}{2}} \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}. \tag{C.3}$$

The gate-level decomposition used in the chapter 5 code (Hadamards plus controlled-phase gates followed by swaps) is precisely the standard factorisation of this matrix into one- and two-qubit gates. Applying QFT_2 followed by $\mathrm{QFT}_2^\dagger$ yields the identity, which is what the `validate_qft_identity` fidelity check verifies numerically.

C.1.3 Three-qubit case ($n = 3$)

For $n = 3$, the QFT is an 8×8 matrix with entries

$$\big[\mathrm{QFT}_3\big]_{k,x} = \frac{1}{\sqrt{8}} \exp\!\left(\frac{2\pi i k x}{8}\right), \qquad k, x \in \{0, \ldots, 7\}. \tag{C.4}$$

Rather than writing all entries explicitly, it is often more informative to view the action on computational-basis states:

$$\mathrm{QFT}_3 \left|x\right\rangle = \frac{1}{\sqrt{8}} \sum_{k=0}^{7} e^{2\pi i k x/8} \left|k\right\rangle. \tag{C.5}$$

The chapter 5 function `qft_on_basis_state` numerically produces these superpositions for specific $|x\rangle$ and compares them for the manual and library implementations via state fidelity.

C.2 Binary-to-phase mapping in QPE

C.2.1 Phase encoding in QPE

In the basic QPE setup, we consider a unitary U and an eigenstate $|u\rangle$ such that

$$U \left|u\right\rangle = e^{2\pi i \phi} \left|u\right\rangle, \qquad \phi \in [0, 1). \tag{C.6}$$

We allocate t counting qubits and prepare them and the target as

$$\left|0\right\rangle^{\otimes t} \left|u\right\rangle \xrightarrow{H^{\otimes t}} \frac{1}{2^{t/2}} \sum_{k=0}^{2^t - 1} \left|k\right\rangle \left|u\right\rangle. \tag{C.7}$$

We then apply controlled powers of U,

$$\prod_{j=0}^{t-1} \left(\mathrm{CU}^{2^j}\right), \tag{C.8}$$

so that the joint state becomes

$$\frac{1}{2^{t/2}} \sum_{k=0}^{2^t-1} e^{2\pi i k\phi} |k\rangle |u\rangle . \tag{C.9}$$

Finally, applying $\mathrm{QFT}_t^\dagger$ to the counting register ideally yields

$$\mathrm{QFT}_t^\dagger \left(\frac{1}{2^{t/2}} \sum_{k=0}^{2^t-1} e^{2\pi i k\phi} |k\rangle \right) \approx |\tilde{\phi}\rangle , \tag{C.10}$$

where $|\tilde{\phi}\rangle$ is a computational-basis state encoding a t-bit approximation to ϕ in binary. The chapter 5 code implements this construction for a simple unitary whose action on $|1\rangle$ is a phase rotation, so the controlled powers of U reduce to controlled-phase gates.

C.2.2 Binary fraction interpretation

Given a t-bit measurement outcome

$$b_{t-1} b_{t-2} \cdots b_1 b_0, \tag{C.11}$$

QPE interprets this as a fixed-point binary fraction:

$$\hat{\phi} = 0.\, b_{t-1} b_{t-2} \cdots b_0 = \sum_{j=0}^{t-1} \frac{b_j}{2^{t-j}} = \frac{\mathrm{int}(b_{t-1} b_{t-2} \cdots b_0)}{2^t}. \tag{C.12}$$

This is exactly what the helper `bitstring_to_phase` does in the chapter 5 code: it converts the bitstring to an integer and divides by 2^t to obtain $\hat{\phi} \in [0, 1)$. When the true phase ϕ is of the form $k/2^t$ for some integer k, and in the absence of noise, QPE outputs the corresponding bitstring with high probability, so $\hat{\phi} = \phi$ up to numerical precision.

C.2.3 Phase error metric

The phase-estimation error in chapter 5 is defined as the wrapped absolute difference

$$\Delta\phi = \min\big(|\hat{\phi} - \phi|,\ 1 - |\hat{\phi} - \phi|\big), \tag{C.13}$$

which accounts for the circular nature of phase modulo 1. This is the value reported as `absolute_error` in each `QPEResult`. In the ideal simulator, $\Delta\phi$ is observed to be very close to zero for phases exactly representable with the chosen number of counting qubits. Under the depolarizing-plus-readout noise model, $\Delta\phi$ increases and the dominant-outcome probability decreases, providing a simple scalar measure of QPE performance degradation in noisy conditions.

Bibliography

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press, 2010.

[2] IBM Quantum Learning, *IBM Quantum Learning Platform*, Available at: `https://learning.quantum.ibm.com/`.

[3] Qiskit Documentation, *Qiskit API Reference and Tutorials*, Available at: `https://qiskit.org/documentation/`.

[4] A. Vichare, *QuantumComputing GitHub Repository*, Available at: `https://github.com/AadityaInnit/QuantumComputing`.

[5] IBM Quantum Learning, *Deutsch's algorithm*, Available at: `https://quantum.cloud.ibm.com/learning/courses/fundamentals-of-quantum-algorithms/quantum-query-algorithms/deutsch-algorithm`.

[6] IBM Quantum Learning, *The Deutsch–Jozsa algorithm*, Available at: `https://quantum.cloud.ibm.com/learning/courses/fundamentals-of-quantum-algorithms/quantum-query-algorithms/deutsch-jozsa-algorithm`.

[7] Wikipedia, *Phase kickback*, Available at: `https://en.wikipedia.org/wiki/Phase_kickback`.

[8] J. Vetterle, *Step by step guide to Deutsch's Algorithm*, Available at: `https://www.jonvet.com/blog/math-behind-deutsch-algorithm`.

[9] Qniverse, *Deutsch–Jozsa Algorithm*, Available at: `https://qniverse.in/docs/deutsch-jozsa-algorithm/`.

[10] IBM Quantum Documentation, *Statevector*, Available at: `https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.quantum_info.Statevector`.

[11] Qiskit Aer Documentation, *Simulators – Qiskit Aer 0.17.1*, Available at: `https://qiskit.github.io/qiskit-aer/tutorials/1_aersimulator.html`.

[12] Qiskit HPC Documentation, *Statevector Simulation*, Available at: https://docs.hpc.gwdg.de/services/quantum-computing/qiskit/index.html.

[13] Q-CTRL, *Quantum Sensing*, Available at: https://q-ctrl.com/quantum-sensing.

[14] Q-CTRL, *How Quantum Sensing Works*, Available at: https://q-ctrl.com/quantum-sensing/how-it-works.

[15] PIRSA, *Quantum Insights into Astronomical Interferometry*, Available at: https://pirsa.org/.

[16] Quantum Astrophysics Group, *Quantum-Assisted Interferometry for Astronomy*, Available at: https://quantum-astronomy.org/.

[17] Nature, *Entanglement-Assisted Non-Local Interferometry*, *Nature*, volume 607, pages 684–689, year 2022.

[18] Physical Review A, *Continuous-Variable Quantum Teleportation for Astronomical Interferometry*, *Physical Review A*, volume 105, number 4, pages 042603, year 2022.

[19] NASA, *Cold Atom Lab: Atom Interferometry in Space*, Available at: https://www.nasa.gov/mission_pages/station/research/experiments/2152.html.

[20] Sandia National Laboratories, *Atom Interferometry for Inertial Sensing*, Available at: https://www.sandia.gov/.

[21] arXiv preprint, *Quantum Sensors: A Perspective*, Available at: https://arxiv.org/abs/quantum-sensors-perspective.

[22] Infleqtion, *Quantum Inertial Sensing*, Available at: https://infleqtion.com/quantum-inertial-sensing.

[23] Springer, *Quantum Sensors for Navigation*, Available at: https://link.springer.com/.

[24] Tech Times, *Quantum Navigation Technologies*, Available at: https://www.techtimes.com/.

[25] Nature, *NV Centers for Many-Body Physics*, *Nature*, volume 580, pages 84–89, year 2020.

[26] Science, *Covariance Spectroscopy with NV Centers*, *Science*, volume 356, pages 832–837, year 2017.

[27] Springer, *Asteroid Resource Prospecting*, Available at: https://link.springer.com/.

[28] Envisioning, *Asteroid Prospecting and Mining*, Available at: https://envisioning.io/.

[29] V. Giovannetti, S. Lloyd, and L. Maccone, *Quantum-enhanced measurements: beating the standard quantum limit*, *Science*, volume 306, number 5700, pages 1330–1336, year 2004.

[30] A. M. Souza, G. A. Alvarez, and D. Suter, *Robust dynamical decoupling for quantum computing and quantum memory*, *Physical Review Letters*, volume 106, number 24, pages 240501, year 2011.

[31] S. Gaudio, *Fundamental Limits of Quantum Sensors for Gravitational Wave Detection*, arXiv preprint arXiv:2603.06772, year 2026.

[32] The Quantum Insider, *China's Space-Based Atom Interferometer Pushes Limits of Inertial Sensing*, Available at: https://thequantuminsider.com/2025/05/16/chinas-space-based-atom-interferometer-pushes-limits-of-inertial-sensing/, year 2025.

[33] A. Kealy et al., *Quantum sensors for enhanced positioning and navigation*, *GPS Solutions*, volume 30, pages 2030, year 2026, 10.1007/s10291-026-02030-y.

[34] Exail, *A leap towards quantum inertial sensing for onboard applications with atom interferometry*, Available at: https://www.exail.com/news/a-leap-towards-quantum-inertial-sensing-for-onboard-applications-with-atom-interf year 2024.

[35] Phys.org, *Quantum sensing via matter-wave interferometry aboard the ISS could broaden our knowledge of the universe*, Available at: https://phys.org/news/2025-05-quantum-interferometry-aboard-iss-broaden.html, year 2025.

[36] S. Abend et al., *Technology roadmap for cold-atoms based quantum inertial sensors in space*, University of Strathclyde, year 2023, Available at: https://strathprints.strath.ac.uk/85041/.

[37] K. Goda, *Squeezed Light for Gravitational Wave Detection*, Ph.D. thesis, Massachusetts Institute of Technology, year 2007, Available at: https://gwic.science/assets/docs/theses/Goda_Thesis.pdf.